CSA17 – ARTIFICIAL INTELLIGENCE

Assignment 4 – CO4

Learning: Algorithm Design, Implementation, Test, Debug and Evaluate

Item	Detail
Student name / Register no.	__P. Lakshmi Gayathri Navaneetha / _192321169_
Nature of assignment	Individual – single integrated assignment (50 marks)
Workflow followed	Design (reasoning, pseudocode, manual trace) → Implement → Instrument → Test → Break → Debug → Validate → Defend
Reproducibility	Python 3.12.3, NumPy 2.5.4, scikit-learn 1.9.1, Matplotlib 3.11.2; global seed 42; every number in this report was produced by the scripts delivered with it

TASK 1 – Pseudocode Review: Diagnose and Repair a Poor Learning Protocol (5 marks)

Task 1.1 – Diagnose the protocol [Design]

The junior engineer's algorithm is: read Data → "train a suitable model" on Data → test on the same Data → print accuracy. Ten deficiencies follow; the last column links each to what the demonstration in Task 1.3 shows.

#	Deficiency	Why it misleads the engineer
1	Testing on the training data (resubstitution estimate)	A flexible model can memorise the data, so training accuracy measures memory, not prediction. Reported 100 % can coexist with poor performance on new students.
2	No held-out test set	There is no data the model has never influenced, so there is no unbiased estimate of future behaviour.
3	No validation / cross-validation for model or hyper-parameter selection	Choices (depth, k, regularisation) made by looking at the score on the same data select the most over-fitted setting. If the test set is used for the choice it is no longer a test set.
4	Accuracy alone on imbalanced data	With ≈ 10 % at-risk students, "nobody is at risk" scores ≈ 90 % accuracy while finding zero at-risk students. Precision, recall and F1 of the at-risk class are needed.
5	No baseline	A number such as 0.88 means nothing until it is compared with the majority-class rule. The model may be worse than doing nothing.
6	No control of randomness (seed, split, initialisation)	Results cannot be reproduced or compared; one lucky split can look like progress.
7	Ambiguous step "Train a suitable model"	It hides the algorithm, loss, hyper-parameters, stopping rule and feature handling – a second engineer cannot implement it and nobody can audit it.
8	Pre-processing (scaling, imputing, feature selection) not separated from evaluation	If statistics are computed on all rows before splitting, information from the evaluation rows leaks into training (data leakage) and inflates the score.
9	A single number with no uncertainty	One accuracy value hides variance between folds/splits; differences of one or two points may be noise.
10	No definition of the positive class, decision threshold or error costs	Missing an at-risk student (false negative) is usually far more costly than a false alarm; the protocol cannot express this, so it optimises the wrong thing.

Task 1.2 – Repair the protocol [Design]

The repaired algorithm separates the data once, reserves a test set that is touched exactly once, selects among **explicitly named** candidate models by stratified K-fold cross-validation on the training part only, compares everything with the majority-class baseline, and reports class-wise metrics.

Pseudocode

```
ALGORITHM TrainValidateTest(D, seed, K, P_test)
INPUT : D = {(x_i, y_i)}, i = 1..n, y_i ∈ {0,1} (1 = at-risk); seed; K = 5; P_test = 0.25
        Candidate set M:  (LogReg, C ∈ {0.01, 0.1, 1, 10}),
                          (DecisionTree, max_depth ∈ {2, 3, 5, 8, none}),
                          (kNN, k ∈ {1, 5, 15})
OUTPUT: chosen model h*, test metrics of h*, test metrics of baseline
BEGIN
 1  RNG ← NewGenerator(seed)
 2  (D_train, D_test) ← StratifiedSplit(D, P_test, RNG)       // class ratio preserved
 3  c0 ← most frequent label in D_train                        // majority-class baseline
 4  (F_1..F_K) ← StratifiedFolds(D_train, K, RNG)
 5  FOR EACH candidate (A, θ) IN M ∪ {Baseline(c0)} DO
 6      FOR j ← 1 TO K DO
 7          fit scaler S on D_train \ F_j ONLY                  // no leakage
 8          h ← Fit(A, θ, S(D_train \ F_j))
 9          m_j ← {Accuracy, Precision_1, Recall_1, F1_1} of h on S(F_j)
10      END FOR
11      score(A, θ) ← mean_j F1_1(m_j);   spread(A, θ) ← std_j F1_1(m_j)
```

```
12  END FOR
13  (A*, θ*) ← argmax score(A, θ) over M       (ties → simpler model, then smaller θ)
14  fit scaler S* and model h* on ALL of D_train with (A*, θ*)
15  evaluate h* ONCE on S*(D_test): confusion matrix, Accuracy, Precision_1, Recall_1, F1_1, ROC-
AUC
16  evaluate Baseline(c0) on D_test with the same metrics
17  IF F1_1(h*) ≤ F1_1(Baseline) THEN REPORT "no skill beyond the baseline"
18  PRINT seed, K, selected (A*, θ*), score ± spread, test metrics, baseline metrics
19  RETURN h*
END
```

Control of randomness is in steps 1, 2 and 4; no test data is used before step 15; the model is never "suitable" – each candidate and each grid value is spelled out; step 7 prevents leakage; the baseline is cross-validated in step 5 and evaluated on the test set in step 16.

Task 1.3 – Demonstrate the defect [Implement]

Test input. Two data sets: (A) a synthetic "at-risk student" set (800 rows, 12 features, 10 % at-risk, 5 % label noise – imbalanced and noisy like real data) and (B) the public Wisconsin breast-cancer data (569 rows, 30 features; positive = malignant). 75 % / 25 % stratified split (seed 42); 5-fold stratified CV on the training part. **Expected behaviour:** training accuracy ≥ cross-validated accuracy ≈ test accuracy for flexible models, and a model with high accuracy but no minority-class skill on set A.

Source code (excerpt)

```
for name, make in models.items():                          # tree (no depth limit), 1-NN, logistic regression
    m = make().fit(X_train, y_train)
    train_acc = accuracy_score(y_train, m.predict(X_train))            # the defective estimate
    cv_acc    = cross_val_score(make(), X_train, y_train, cv=skf).mean()   # repaired: 5-fold CV
    test_acc  = accuracy_score(y_test,  m.predict(X_test))             # repaired: held-out, used once
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
```

Actual output (accuracy; "gap" = training accuracy − held-out test accuracy = the optimism of the defective protocol):

Data	Model	Train acc.	5-fold CV acc.	Test acc.	Gap	Test F1 (positive)
A (imbalanced)	Majority-class baseline	0.883	0.883	0.885	-0.002	0.000
A (imbalanced)	Decision tree (unlimited depth)	1.000	0.833	0.825	+0.175	0.146
A (imbalanced)	1-nearest-neighbour	1.000	0.848	0.850	+0.150	0.250
A (imbalanced)	Logistic regression	0.897	0.893	0.875	+0.022	0.000
B (breast cancer)	Majority-class baseline	0.627	0.627	0.629	-0.003	0.000
B (breast cancer)	Decision tree (unlimited depth)	1.000	0.916	0.958	+0.042	0.941
B (breast cancer)	1-nearest-neighbour	1.000	0.953	0.937	+0.063	0.913
B (breast cancer)	Logistic regression	0.988	0.972	0.965	+0.023	0.951

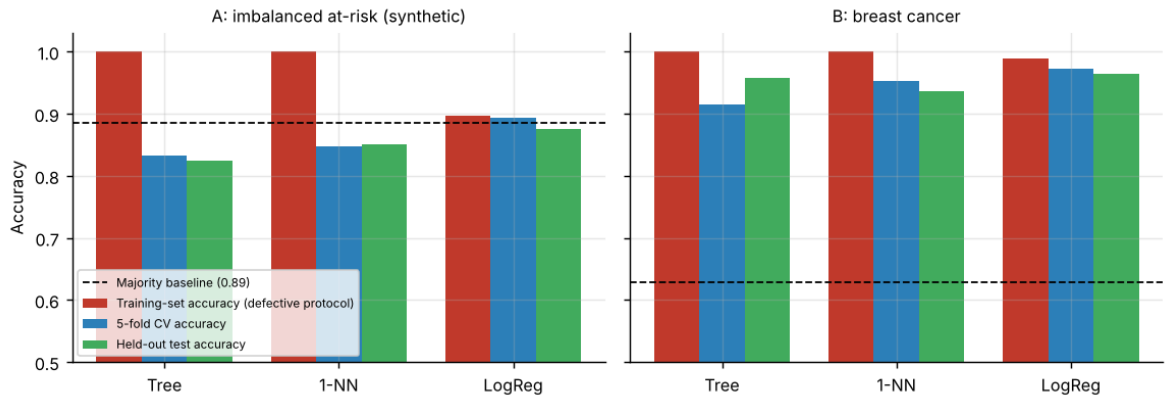


Figure 1.1 – The defective protocol (red) reports a much higher accuracy than cross-validation (blue) or the held-out set (green). The dashed line is the majority-class baseline.

> **Interpretation.** For the unlimited-depth tree on the imbalanced data the defective protocol prints **1.000**; 5-fold CV gives 0.833 and the held-out test set 0.825 – an optimistic bias of **17.5 accuracy points**. It is worse than that: the majority-class baseline already reaches 0.885 test accuracy, so the “perfect” tree is actually *below* the do-nothing rule on accuracy, and its test F1 for the at-risk class is only 0.146. On the cleaner breast-cancer data the optimism is smaller (tree 1.000 → 0.958, 1-NN 1.000 → 0.937) but still always positive for memorising models. Logistic regression, a low-variance model, shows a small gap – the defect hurts most exactly for flexible models, which are the ones engineers are tempted to use.

Boundary test. The gap of the baseline is ≈ 0 (it cannot memorise) – confirming that the gap measures over-fitting and not a bug in the evaluation code. All splits use `random_state = 42`, so the table is reproducible.

TASK 2 – From Requirement to Pseudocode to Working Model: Decision Trees (9 marks)

Task 2.1 – Model the requirement [Design]

Element	Specification
Input	A student described by three categorical attributes: Attendance ∈ {High, Low}, Assignments ∈ {Done, Missed}, Internal Marks ∈ {Good, Poor}.
Output	A class label Result ∈ {Pass, Fail} (binary classification).
Objective	Learn a function from the 8 labelled examples that classifies them correctly and is as small as possible, so that it generalises to unseen students and can be read by a tutor (early-warning rule).
Hypothesis space	All finite decision trees whose internal nodes test exactly one categorical attribute with one branch per attribute value and whose leaves carry a class label.
Constraints	Deterministic classification; an attribute is tested at most once on any root-to-leaf path; every path must end in a leaf; no numeric thresholds are needed here; the tree must be built only from the training examples.
Decision at a node	Which not-yet-used attribute to test next for the examples that reached the node – or whether to stop and return a leaf.
Information needed to choose	The class counts of the examples at the node, and for every candidate attribute the class counts inside each of its value-partitions (so that the uncertainty before and after the split can be compared).

Task 2.2 – Splitting hypothesis [Design]

> **Rule:** at every node test the attribute whose value-partition gives the largest reduction in class entropy, $\text{Gain}(E, a) = H(E) - \Sigma_v (|E_v|/|E|) \cdot H(E_v)$, breaking ties in favour of the attribute listed first.

Justification. Entropy measures how many further yes/no questions are still needed to name the class. The greedy choice removes the most uncertainty per test, so children become pure sooner, fewer further tests are required and (Occam's razor) the resulting tree tends to be short, which in turn tends to generalise better than a long tree that merely memorises the examples.

Task 2.3 – Design the algorithm [Design]

Pseudocode

```
ALGORITHM BuildTree(E, A, defaultClass)
INPUT : E – set of examples (attribute values, label); A – attributes still available;
        defaultClass – majority class of the parent node
OUTPUT: decision tree (a Leaf or an InternalNode with one child per attribute value)
BEGIN
 1  IF E is empty THEN RETURN Leaf(defaultClass)                  // stop 1: no examples
 2  IF every example in E has the same label c THEN RETURN Leaf(c) // stop 2: pure node
 3  IF A is empty THEN RETURN Leaf(MajorityLabel(E))               // stop 3: no attributes left
 4  m ← MajorityLabel(E)
 5  FOR EACH attribute a IN A DO
 6      g[a] ← InformationGain(E, a)
 7  END FOR
 8  a* ← attribute with the largest g[a]   (tie → the one that comes first in the schema)
 9  node ← InternalNode(test = a*)
10  FOR EACH value v of a* that occurs in E DO
11      E_v ← { e ∈ E : e.a* = v }                                  // partition
12      node.child[v] ← BuildTree(E_v, A \ {a*}, m)                 // recursion on a smaller
problem
13  END FOR
14  RETURN node
END

FUNCTION InformationGain(E, a)
    remainder ← 0
    FOR EACH value v of a occurring in E DO
        E_v ← { e ∈ E : e.a = v };   remainder ← remainder + (|E_v| / |E|) · Entropy(E_v)
    END FOR
    RETURN Entropy(E) – remainder
```

```
FUNCTION Entropy(E)
    H ← 0;  FOR EACH class c DO  p ← count(c in E) / |E|;  IF p > 0 THEN H ← H − p · log2(p)  END
FOR
    RETURN H
```

Termination: every recursive call removes one attribute from `A` (or has fewer examples), so recursion depth ≤ |A| = 3. For **numeric** attributes (used in Task 2.5–2.6) step 5–8 additionally scan every mid-point between consecutive sorted values as a binary threshold `x ≤ t`, and the attribute is not removed from `A`.

Task 2.4 – Desk-check [Design]

Training set: 8 examples, 4 Pass / 4 Fail. Using $\log_2$ and $H(p) = -p \log_2 p - (1-p)\log_2(1-p)$: **H(root) = H(4/8) = 1.0000**; H(3/4) = 0.8113; H(1/2) = 1; H(0) = 0.

Root node (all 8 rows)

Attribute	Partition (rows) → Pass/Fail	Entropy of parts	Remainder	Gain
Attendance	High (1–4): 3P/1F · Low (5–8): 1P/3F	0.8113 , 0.8113	½·0.8113 + ½·0.8113 = 0.8113	0.1887
Assignments	Done (1,2,5,6): 3P/1F · Missed (3,4,7,8): 1P/3F	0.8113 , 0.8113	0.8113	0.1887
Internal Marks	Good (1,3,5,7): 3P/1F · Poor (2,4,6,8): 1P/3F	0.8113 , 0.8113	0.8113	0.1887

All three gains are equal (0.1887): the data are the "majority-of-three" function (Pass ⇔ at least two of the three indicators are favourable), which is symmetric in the attributes. By the tie-break rule the root is **Attendance**.

Branch Attendance = High (rows 1–4: P, P, P, F; H = 0.8113)

Attribute	Partition	Remainder	Gain
Assignments	Done {1,2}: P,P (H=0) · Missed {3,4}: P,F (H=1)	½·0 + ½·1 = 0.5	0.3113
Internal Marks	Good {1,3}: P,P (H=0) · Poor {2,4}: P,F (H=1)	0.5	0.3113

Tie → **Assignments**. Done → pure Pass (stop 2). Missed → rows 3, 4 are P/F; only Internal Marks remains, Gain = 1 − 0 = 1.000 → Good → Pass, Poor → Fail.

Branch Attendance = Low (rows 5–8: P, F, F, F; H = 0.8113)

Attribute	Partition	Remainder	Gain
Assignments	Done {5,6}: P,F (H=1) · Missed {7,8}: F,F (H=0)	½·1 + ½·0 = 0.5	0.3113
Internal Marks	Good {5,7}: P,F (H=1) · Poor {6,8}: F,F (H=0)	0.5	0.3113

Tie → **Assignments**. Missed → pure Fail. Done → rows 5, 6 split by Internal Marks (Gain = 1): Good → Pass, Poor → Fail.

Resulting tree (printed by the implementation, [Fail, Pass] counts at leaves) – identical to the hand trace

```
Attendance = High:
    Assignments = Done:
        -> Pass  [0, 2]
    Assignments = Missed:
        InternalMarks = Good:
            -> Pass  [0, 1]
        InternalMarks = Poor:
            -> Fail  [1, 0]
Attendance = Low:
    Assignments = Done:
        InternalMarks = Good:
            -> Pass  [0, 1]
        InternalMarks = Poor:
            -> Fail  [1, 0]
    Assignments = Missed:
        -> Fail  [2, 0]
```

The code reproduces every gain printed above (0.1887, 0.3113, 1.0000) and the same tree: depth 3, 6 leaves, training accuracy 8/8.

Task 2.5 – Implement and verify [Implement]

Implementation (`dtree.py`): recursive learner with entropy or Gini criterion, multi-way splits for categorical attributes, best-threshold search for numeric attributes (cumulative class counts over the sorted values, mid-point thresholds), tie-breaks by feature order. **Data:** Wisconsin breast cancer (569 × 30, positive class = malignant), 70 % / 30 % stratified split, seed 42 – the same split is given to scikit-learn's `DecisionTreeClassifier` (`random_state = 42`, no depth limit).

Source code (excerpt)

```
def _best_split(self, idx, attrs, counts, path):          # numeric branch (condensed)
    parent = self.imp(counts); n = len(idx); best = None
    for j in attrs:
        order = np.argsort(self.X[idx, j]); xs = self.X[idx, j][order]; ys = self.y[idx][order]
        left = np.cumsum(one_hot(ys), axis=0)[:-1]; right = counts - left     # class counts left/right
        nl = np.arange(1, n); nr = n - nl
        valid = (xs[1:] > xs[:-1]) & (nl >= self.min_samples_leaf) & (nr >= self.min_samples_leaf)
        gains = parent - (nl * impurity(left) + nr * impurity(right)) / n    # information gain / Gini
decrease
        k = argmax(where(valid, gains, -inf))
        if best is None or gains[k] > best[0] + 1e-12:
            best = (gains[k], j, (xs[k] + xs[k + 1]) / 2)                  # mid-point threshold
    return best
```

Actual output – test-set metrics (171 test students; precision/recall/F1 for the malignant class):

Criterion / implementation	Accuracy	Precision	Recall	F1	Depth	Leaves
Entropy – from scratch	0.953	0.983	0.891	0.934	8	16
Entropy – scikit-learn	0.942	0.966	0.875	0.918	8	16
Gini – from scratch	0.901	0.898	0.828	0.862	8	19
Gini – scikit-learn	0.901	0.885	0.844	0.864	8	19

Confusion matrix [[TN, FP],[FN, TP]]	From scratch	scikit-learn
Entropy	[[106, 1], [7, 57]]	[[105, 2], [8, 56]]
Gini	[[101, 6], [11, 53]]	[[100, 7], [10, 54]]

Verification. Both implementations choose the same root ("worst perimeter", threshold 112.8 vs 112.8), grow trees of identical depth and number of leaves, and agree on **97.7%** of the 171 test predictions (entropy) and 97.7% (Gini): 4 predictions differ. The differences come from deeper nodes where several candidate splits have exactly equal gain; scikit-learn breaks such ties through a random feature permutation, mine by feature order. Metrics therefore differ by at most one or two test students (e.g. entropy accuracy 0.953 vs 0.942); this is tie-breaking noise, not a logic error. With Gini the from-scratch accuracy equals scikit-learn's exactly (0.901).

Task 2.6 – Overfitting and pruning [Implement]

Data are re-split 60 % train / 20 % validation / 20 % test (stratified, seed 42). The full tree is grown, then depth and minimum-leaf-size limits and **reduced-error pruning** (bottom-up: replace a subtree by its majority leaf whenever validation errors do not increase) are applied. Accuracy is reported on the three sets.

Setting	Depth	Leaves	Train acc.	Validation acc.	Test acc.
full tree (no limit)	5	14	1.000	0.939	0.939
max_depth=1	1	2	0.930	0.930	0.877
max_depth=2	2	4	0.938	0.904	0.939
max_depth=3	3	7	0.977	0.947	0.947
max_depth=4	4	11	0.991	0.939	0.939

Setting	Depth	Leaves	Train acc.	Validation acc.	Test acc.
max_depth=5	5	14	1.000	0.939	0.939
max_depth=6	5	14	1.000	0.939	0.939
min*samples*leaf=5	4	11	0.979	0.939	0.956
min*samples*leaf=10	4	8	0.971	0.947	0.930
min*samples*leaf=20	4	6	0.938	0.904	0.939
reduced-error pruning	3	6	0.977	0.947	0.947

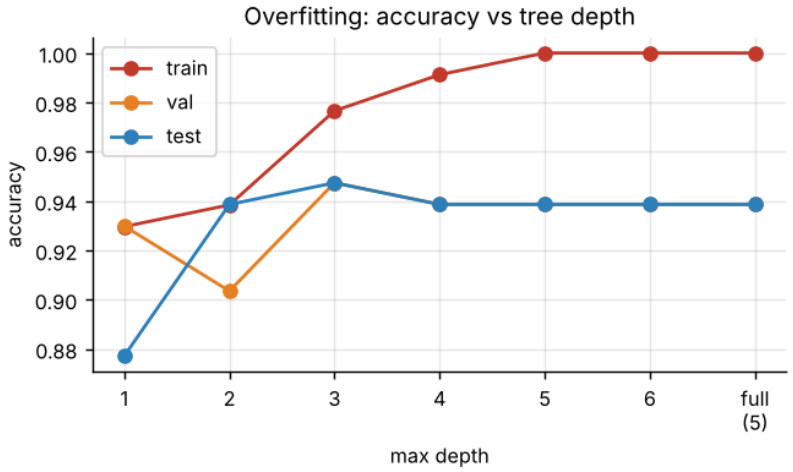


Figure 2.1 – Training accuracy rises monotonically with depth; validation/test accuracy peak at depth 3 and then stay flat or fall.

> **Effect.** The unrestricted tree reaches 100 % training accuracy with 14 leaves but only 0.939 on validation and test data. Limiting depth to 3 (7 leaves) or applying reduced-error pruning (6 leaves, depth 3) gives lower training accuracy (0.977) but equal or better validation and test accuracy (0.947) with less than half the leaves: the dropped splits fitted individual training examples, not structure. Depth 1 under-fits (0.877 test). Caution: the test set has only 114 students, so one student = 0.9 percentage points; the evidence is "pruning does not hurt and the tree becomes much simpler", not a precise ranking.

Task 2.7 – Challenge the design

(a) Change one value in the desk-check table

Change: Row 8, Attendance Low → **High** (row 8 becomes High / Missed / Poor → Fail).

Prediction before tracing. The attributes Assignments and Internal Marks are untouched: their partitions still contain 3P/1F and 1P/3F, so their gain stays 0.1887. Attendance now splits the rows as High {1,2,3,4,8} = 3P/2F (H = 0.9710) and Low {5,6,7} = 1P/2F (H = 0.9183), remainder = 5/8·0.9710 + 3/8·0.9183 = 0.9513, so Gain(Attendance) = 0.0487 < 0.1887. **I therefore predict that the root changes from Attendance to Assignments** (tie with Internal Marks, broken by column order).

Verification by trace and by code. Hand calculation gives Gain(Attendance) = 0.0487, Gain(Assignments) = Gain(Internal Marks) = 0.1887. The program prints Gain(Attendance) = 0.0488, Gain(Assignments) = 0.1887, Gain(Internal Marks) = 0.1887 – the prediction is **confirmed**; the new root is Assignments and the tree (6 leaves) is below.

Tree after modifying Row 8

```
Assignments = Done:
    Attendance = High:
        -> Pass  [0, 2]
    Attendance = Low:
        InternalMarks = Good:
            -> Pass  [0, 1]
        InternalMarks = Poor:
            -> Fail  [1, 0]
Assignments = Missed:
```

```
InternalMarks = Good:
    Attendance = High:
        -> Pass  [0, 1]
    Attendance = Low:
        -> Fail  [1, 0]
InternalMarks = Poor:
    -> Fail  [2, 0]
```

Lesson: a single changed value in one row is enough to reorder the attribute ranking and restructure the whole tree – decision trees are unstable (high variance), which is one reason ensembles and pruning are used.

(b) Identifier-like attribute: the bias of information gain

Add a Roll-No attribute with a unique value for each of the 8 rows. Every partition then contains one example and is pure, so remainder = 0 and **Gain(RollNo) = H(root) = 1.000**, far above 0.1887 for the real attributes: plain information gain picks Roll-No as root and builds an 8-leaf tree that is a lookup table – it classifies a new student by an ID the tree has never seen. Information gain is biased toward attributes with many values.

Attribute	Gain	Split information	Gain ratio
RollNo	1.0000	3.000	0.3333
Attendance	0.1887	1.000	0.1887
Assignments	0.1887	1.000	0.1887
InternalMarks	0.1887	1.000	0.1887

Repair 1 – gain ratio (C4.5): GainRatio = Gain / SplitInfo, SplitInfo = $-\Sigma\,(|E_v|/|E|)\,\log_2(|E_v|/|E|) = \log_2 8 = 3$ for the ID. On this 8-row toy it gives 0.333, which is **still larger than 0.1887, so gain ratio alone does not repair the toy case** (the root remains Roll-No, 8 leaves). To see whether the repair works when the data are not tiny, I generated 300 noisy students (majority-of-three rule, 10 % label noise) plus a unique ID, and 1000 fresh students whose IDs the tree has never seen:

Variant	Root chosen	Leaves	Train acc.	Test acc. (unseen IDs)
Without ID	InternalMarks	8	0.900	0.891
With ID, information gain	ID	300	1.000	0.481
With ID, gain ratio	InternalMarks	300	1.000	0.891

Result. With n = 300 the ID has Gain = 0.987 (maximal) but Gain ratio only 0.120, below InternalMarks (0.135), so gain ratio now keeps the root sensible. Plain gain builds a 300-leaf memoriser whose accuracy on unseen students is **0.481 – below chance and below the 0.891 of the tree without the ID**. Gain ratio rescues the root, but the tree still has 300 leaves because deeper nodes contain few examples, where the ID's ratio becomes competitive again; accuracy on unseen IDs recovers only because unseen values fall back to the node majority. **Complete repair = three measures together:** (i) exclude identifier-like attributes using a cardinality test (distinct values / n ≈ 1) or domain knowledge – the real fix; (ii) gain ratio, preferably with C4.5's rule "only consider attributes with at least average gain"; (iii) a minimum number of examples per leaf (or validation-based pruning), which forbids one-example leaves.

TASK 3 – Pseudocode Forensics: Hidden Variables and the EM Algorithm (6 marks)

Task 3.1 – Locate the defect [Design]

For a mixture $p(x) = \Sigma_k \pi_k N(x \mid \mu_k, \sigma^2_k)$ the hidden variable z_i says which component generated x_i. The E-step must compute the **posterior** responsibility $\gamma_{ik} = P(z_i = k \mid x_i) = \pi_k N(x_i|\mu_k,\sigma^2_k) / \Sigma_j \pi_j N(x_i|\mu_j,\sigma^2_j)$.

> **Defect:** the statement `r[i][k] ← pi[k] * Normal(X[i] | mu[k], var[k])` is incomplete – the division by the row total $\Sigma_j \pi_j N(x_i|\mu_j,\sigma^2_j)$ is missing. The stored quantity is the **joint** density $p(x_i, z_i = k)$, not the posterior. It no longer has the meaning "probability that point i belongs to component k": Σ_k r[i][k] ≠ 1.

Consequences: $Nk = \Sigma_i$ *r[i][k] is not the expected number of points of component k* ($\Sigma_k Nk \neq n$); $\pi k = Nk / n$ therefore do not sum to 1 (they shrink by the average density at every iteration); and μk, $\sigma^2 k$ become **density-weighted** averages in which points in dense regions count more than points in sparse regions, whatever component they belong to.

Task 3.2 – Failing input and state trace [Design]

My dataset: X = {1, 2, 8, 9} (n = 4, two obvious clusters), K = 2, initial π = (0.5, 0.5), μ = (2, 8), σ^2 = (1, 1).
Prediction made beforehand: the first wrong value is r[1][1] (≈ 0.12 instead of ≈ 1), Σπ will drop far below 1 after one iteration.

i	x_i	r[i][1] (as coded)	r[i][2] (as coded)	row sum (must be 1)	correct γ_{i1}	correct γ_{i2}
1	1	1.2099e-1	4.567e-12	0.1210	1.0000	3.78e-11
2	2	1.9947e-1	3.038e-9	0.1995	1.0000	1.52e-8
3	8	3.0379e-9	1.995e-1	0.1995	0.0000	1.00e+0
4	9	4.5674e-12	1.210e-1	0.1210	0.0000	1.00e+0

Example for i = 1: N(1 | 2, 1) = 0.24197, so r[1][1] = 0.5 · 0.24197 = **0.1210**; r[1][2] = 0.5 · N(1 | 8, 1) ≈ $4.6 \cdot 10^{-12}$. The row sum is 0.1210, not 1. **This is the first incorrect value.** Continuing the M-step with these numbers:

Quantity	Faulty (as coded)	Correct (normalised r)
N_1 , N_2	0.3205 , 0.3205 (sum 0.6409 ≠ n = 4)	2.0000 , 2.0000 (sum 4)
μ_1 , μ_2	1.6225 , 8.3775	1.5000 , 8.5000 (true cluster means 1.5, 8.5)
σ^2_1 , σ^2_2	0.2350 , 0.2350	0.2500 , 0.2500 (true 0.25)
π_1 , π_2	0.0801 , 0.0801	0.5000 , 0.5000
Σ π	0.1602	1.0000

The faulty means (1.62, 8.38) are pulled toward the points with larger density (x = 2 and x = 8 weigh 0.199 vs 0.121 for x = 1 and 9); the variance is 6 % too small; and Σπ = 0.16. The prediction is confirmed.

Task 3.3 – Repair [Design]

Only the E-step changes: after the double loop that fills `r`, one normalisation loop is inserted (shown in bold in the corrected excerpt); M-step and everything else stay untouched.

Corrected E-step (lines marked + are the only change)

```
    FOR i ← 1 TO n DO
        FOR k ← 1 TO K DO
            r[i][k] ← pi[k] * Normal(X[i] | mu[k], var[k])
        END FOR
+       total ← SUM over k of r[i][k]                // ← inserted
+       FOR k ← 1 TO K DO  r[i][k] ← r[i][k] / total  END FOR   // ← inserted
    END FOR
```

After the repair Σ_k r[i][k] = 1 for every i, N_k sums to n, $\Sigma\pi = 1$ holds by construction, and the M-step formulas are exactly the maximum-likelihood updates for the expected complete-data log-likelihood, which gives the monotone-likelihood guarantee of EM.

Task 3.4 – Implement faulty and repaired versions [Implement]

Source code (excerpt)

```
def em(X, pi, mu, var, T, repaired):
    for t in range(T):
        r = np.array([[pi[k]*normal(x, mu[k], var[k]) for k in range(K)] for x in X])   # E-step, as in
the pseudocode
        if repaired:                                    # <-- the only logical difference
            r = r / r.sum(axis=1, keepdims=True)
        for k in range(K):                              # M-step (identical in both versions)
            Nk = r[:, k].sum(); mu[k] = (r[:, k]*X).sum()/Nk
            var[k] = max((r[:, k]*(X-mu[k])**2).sum()/Nk, 1e-6)   # numerical floor, both versions
            pi[k] = Nk/n
        record loglik(X, pi, mu, var) = sum_i log sum_k pi_k N(x_i | mu_k, var_k)  and  sum(pi)
```

Test input: 200 points from a two-component mixture (120 from N(0, 1^2), 80 from N(5, 1.2^2)); **25 random initialisations** (two random data points as initial means, σ^2 = sample variance, π = ½), 60 iterations each. The log-likelihood is always evaluated with the *current* parameters, so a drop in the faulty version is a real drop of the objective.

Measure (25 initialisations)	Faulty	Repaired
Runs in which the log-likelihood decreased at least once	25 / 25	0 / 25
Decreasing iterations per run (of 60)	60.0	0.0
Largest single-step change of log-likelihood	-1949.4	≥ 0 (min = 0.0)
Final log-likelihood, mean ± sd	-26383 ± 602	-402.671 ± 0.000
$\Sigma\pi$ after the first iteration (mean)	0.0806	1.0000
$\Sigma\pi$ after 60 iterations (max over runs)	1.6e-36	1.000000
Estimated means (mean over runs)	0.160 , 1.380 (sd of 2nd = 2.16)	-0.075 , 4.982 (sd ≈ 1e-11)

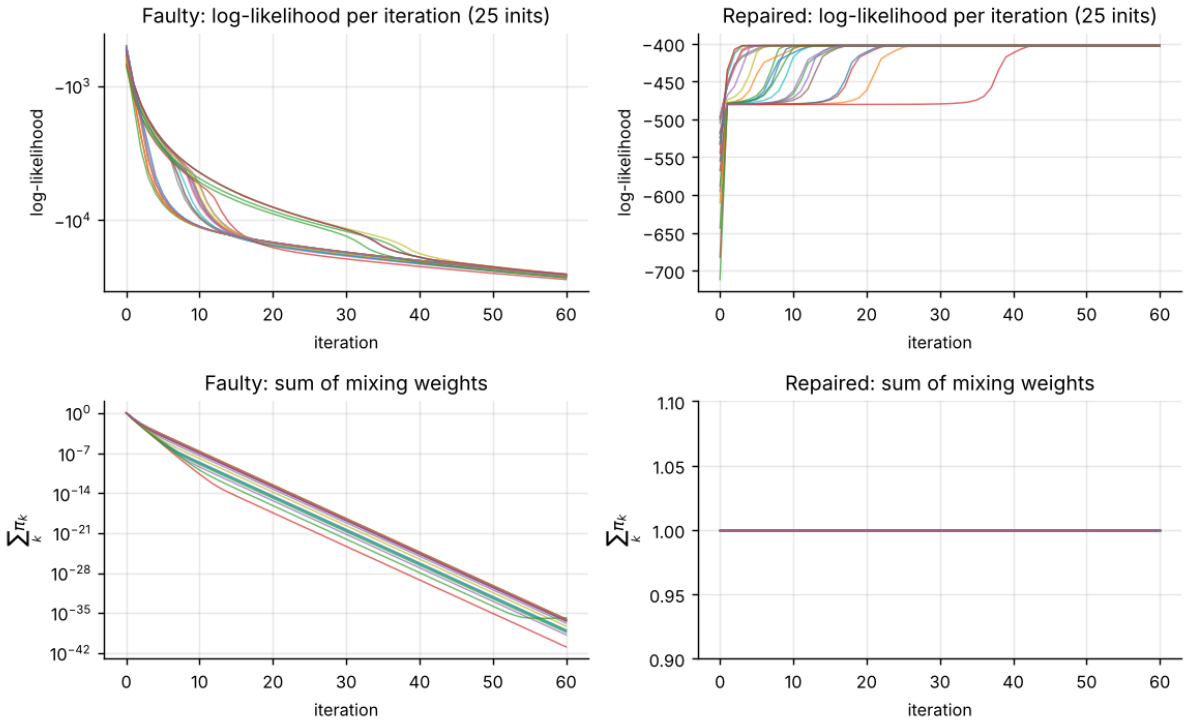


Figure 3.1 – Top: log-likelihood per iteration for 25 initialisations (faulty: symlog axis, falls without bound; repaired: monotone increase to the same optimum). Bottom: $\Sigma\pi$ (faulty: log axis, decays geometrically, about ×0.2 per step; repaired: exactly 1).

Interpretation. The repaired version never decreased the log-likelihood in any of 25 runs and all runs reach the same optimum (−402.671, means -0.075 and 4.982; scikit-learn's `GaussianMixture` gives -0.074 and 4.983, log-likelihood -402.671 – the small difference is its looser stopping tolerance). The faulty version decreased the likelihood in **every iteration of every run**: $\Sigma\pi$ is multiplied by the average component density (≈ 0.08–0.2) each step, so it vanishes after ~60 steps, and the "model" describes a distribution with total mass ≈ 10^{-35}. Note also that the faulty version does not crash – it returns plausible-looking numbers, which is what makes this defect dangerous.

Design note. A variance floor of 10^{-6} is used in both versions to avoid a singular component; it is a numerical safeguard, not part of the logical repair.

Task 3.5 – Regression check

Input on which the original already works. The defect is silent exactly when $\Sigma_k \pi_k N(x_i|\mu_k,\sigma^2_k) = 1$ for every point, because then the missing division changes nothing. Such an input: X = {−a, a, 10−a, 10+a} with a = 0.5·e^(−½)/√(2π) = 0.12099, started at π = (½, ½), μ = (0, 10), σ^2 = (a^2, a^2) – each point then has density exactly 2 under its own component, so πN = 1 (the other component contributes ≈ 10^{-1500}) and these parameters are also the maximum-likelihood fixed point.

Check (10 iterations)	Result
Row sums of the original (un-normalised) r	1.000000 , 1.000000 , 1.000000 , 1.000000
Max |difference| in π, μ, σ^2 between original and repaired	3.2e-14
Max |difference| in log-likelihood over all iterations	1.3e-13
Σπ original / repaired after 10 iterations	1.000000 / 1.000000
Repaired vs scikit-learn on the 200-point data: means / log-lik.	-0.0746, 4.9816 vs -0.0739, 4.9833 ; -402.671 vs -402.671

Conclusion. On this input original and repaired agree to ~10^{-14} (floating-point level), so the repair preserves correct behaviour; on ordinary data the repaired code also agrees with an independent reference implementation. The case also shows why the bug can survive casual testing: it hides on inputs where the normaliser happens to be 1.

TASK 4 – From Recurrence to Executable Logic: Neural Network Back-Propagation (9 marks)

Task 4.1 – Define the state [Design]

Network with layers l = 1 … L; n_l units in layer l; n_0 inputs; f is the hidden activation.

Symbol	Meaning	Dimension
$a^{(0)} = x$	input vector (activation of "layer 0")	$n_0 \times 1$
$W^{(l)}$, $b^{(l)}$	weights and biases of layer l	$n_l \times n_{(l-1)}$, $n_l \times 1$
$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$	pre-activation (weighted input) of layer l	$n_l \times 1$
$a^{(l)} = f(z^{(l)})$	activation (output) of layer l; $a^{(L)} = \hat{y}$ is the network output	$n_l \times 1$
$\delta^{(l)} = \partial E / \partial z^{(l)}$	error term of layer l: sensitivity of the loss to the pre-activation	$n_l \times 1$
$\partial E/\partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$, $\partial E/\partial b^{(l)} = \delta^{(l)}$	gradients used by the update	$n_l \times n_{(l-1)}$, $n_l \times 1$

Task 4.2 – Forward and backward recurrences [Design]

Forward recurrence: $a^{(0)} = x$; for l = 1…L: $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$, $a^{(l)} = f(z^{(l)})$ (output layer: sigmoid for the squared-error network, softmax for classification).

Boundary condition at the output layer (chain rule through the output activation): $\delta^{(L)} = \partial E/\partial a^{(L)} \odot f'(z^{(L)})$. For $E = \tfrac{1}{2}\|a^{(L)} - y\|^2$ with sigmoid output: $\delta^{(L)} = (a^{(L)} - y) \odot a^{(L)} \odot (1 - a^{(L)})$. For softmax with cross-entropy the f′ term cancels and $\delta^{(L)} = a^{(L)} - y$.

Backward recurrence. $z^{(l+1)}$ depends on $z^{(l)}$ only through $a^{(l)} = f(z^{(l)})$, so for unit j of layer l:

$$\delta_j^{(l)} = \partial E/\partial z_j^{(l)} = \Sigma_k \, (\partial E/\partial z_k^{(l+1)}) \cdot (\partial z_k^{(l+1)}/\partial a_j^{(l)}) \cdot (\partial a_j^{(l)}/\partial z_j^{(l)}) = \Sigma_k \, \delta_k^{(l+1)} \, W_{kj}^{(l+1)} \, f'(z_j^{(l)}),$$ i.e. $\boldsymbol{\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot f'(z^{(l)})}$, l = L−1, …, 1.

- $\boldsymbol{(W^{(l+1)})^T \delta^{(l+1)}}$ routes the blame of the next layer back along the same weights that carried the signal forward; each unit receives the weighted sum of the errors of the units it fed.
- $\boldsymbol{f'(z^{(l)})}$ says how strongly a change of z shows up in the activation – ≤ ¼ for sigmoid (source of vanishing gradients), 0 or 1 for ReLU (source of dead units).
- **Why last to first:** $\delta^{(l)}$ is defined through $\delta^{(l+1)}$, so the recurrence can only be evaluated starting from the layer where the loss is known (the output) – dynamic programming: each δ is computed once and reused for the weight gradient of its layer, giving all gradients for the cost of about two forward passes. The forward values $a^{(l-1)}$, $z^{(l)}$ must be stored.

Task 4.3 – Pseudocode for mini-batch training [Design]

Pseudocode

```
ALGORITHM TrainMLP(X, Y, sizes, f, loss, η, B, E_max, tol, seed)
INPUT : X (n × n0), Y (n × n_L); sizes = [n0, …, n_L]; hidden activation f; loss ∈
{SquaredError+Sigmoid, CrossEntropy+Softmax}
        learning rate η; batch size B; max epochs E_max; tolerance tol; seed
OUTPUT: W(1..L), b(1..L)
BEGIN
 1  RNG ← NewGenerator(seed)
 2  FOR l ← 1 TO L DO
 3      s2 ← 2 / n(l-1) IF f is ReLU ELSE 1 / n(l-1)                  // He / Xavier variance
 4      W(l) ← matrix n(l) × n(l-1) of Normal(0, s2) samples;  b(l) ← zeros(n(l))
 5  END FOR
 6  previous ← +∞
 7  FOR epoch ← 1 TO E_max DO
 8      order ← RandomPermutation(1..n)
 9      FOR start ← 1 TO n STEP B DO
10          idx ← order[start .. MIN(start + B − 1, n)];  m ← |idx|;  a(0) ← X[idx]
11          FOR l ← 1 TO L DO                                          // forward pass
12              z(l) ← a(l-1) · W(l)ᵀ + b(l)
```

```
13              a(l) ← f(z(l))  IF l < L  ELSE OutputActivation(z(L))
14          END FOR
15          batchLoss ← MEAN over idx of Loss(a(L), Y[idx])
16          δ(L) ← a(L) − Y[idx]  (softmax+CE)   OR   (a(L) − Y[idx]) ⊙ a(L) ⊙ (1 − a(L))
(sigmoid+SE)
17          FOR l ← L − 1 DOWN TO 1 DO  δ(l) ← (δ(l+1) · W(l+1)) ⊙ f'(z(l))  END FOR     //
backward pass
18          FOR l ← 1 TO L DO                                                  // update
19              gW ← δ(l)ᵀ · a(l−1) / m;   gb ← column sums of δ(l) / m
20              W(l) ← W(l) − η · gW;      b(l) ← b(l) − η · gb
21          END FOR
22      END FOR
23      epochLoss ← MEAN loss over all n examples
24      IF epochLoss is not finite THEN REPORT "diverged"; STOP
25      IF |previous − epochLoss| < tol THEN STOP                       // stopping criterion
26      previous ← epochLoss
27  END FOR
28  RETURN W, b
END
```

Task 4.4 – Desk-check [Design]

Network 2-2-1, sigmoid $\sigma(z) = 1/(1+e^{-z})$, x = (1, 0), y = 1, hidden weights: unit 1 (0.5, −0.4), unit 2 (0.3, 0.8); output weights (0.6, −0.2); all biases 0; $E = \tfrac{1}{2}(a_2 - y)^2$; $\eta = 0.5$. Values to 5 decimals (computed with explicit scalar arithmetic, independent of the matrix code).

Step	Calculation	Result
Forward – hidden z	$z_1 = 0.5\cdot1 + (-0.4)\cdot0$; $z_2 = 0.3\cdot1 + 0.8\cdot0$	z = (0.50000, 0.30000)
Forward – hidden a	a = σ(z)	a = (0.62246, 0.57444)
Forward – output	$zo = 0.6\cdot a_1 - 0.2\cdot a_2$; $\hat{y} = \sigma(zo)$	z_o = 0.25859 ; ŷ = 0.56429
Loss	$E = \tfrac{1}{2}(\hat{y} - 1)^2$	E = 0.09492
Output error	$\delta_o = (\hat{y} - y)\,\hat{y}\,(1 - \hat{y})$	δ_o = -0.10713
Hidden errors	$\delta_j = \delta_o \cdot w_j \cdot a_j(1 - a_j)$	δ = (-0.01511, 0.00524)
Gradients – output layer	$\partial E/\partial w_j = \delta o\, a_j$; $\partial E/\partial bo = \delta_o$	(-0.06668, -0.06154) ; -0.10713
Gradients – hidden layer	$\partial E/\partial W_{ji} = \delta_j x_i$ ($x_2 = 0 \Rightarrow$ column 2 is 0) ; $\partial E/\partial b_j = \delta_j$	row 1: (-0.01511, 0) ; row 2: (0.00524, 0)
Update (η = 0.5) – output	w ← w − 0.5·grad ; bo ← $0 - 0.5\cdot\delta o$	w = (0.63334, -0.16923) ; b_o = 0.05356
Update – hidden	W ← W − 0.5·grad ; b ← −0.5 δ	W_1 = (0.50755, −0.40000) ; W_2 = (0.29738, 0.80000) ; b = (0.00755, -0.00262)
Check – new forward pass	recompute with new parameters	ŷ = 0.58735 ; E = 0.08514 (was 0.09492)

The loss decreases from 0.09492 to 0.08514 after one step and the output moves from 0.564 toward the target 1, as it should. Biases start at 0 but are parameters and receive updates too (δ).

Task 4.5 – Implement and verify [Implement]

Source code (excerpt)

```
def forward(self, X):                       # a[l] = f(z[l]),  z[l] = a[l-1] W[l]^T + b[l]
    a = [X]; z = [None]
    for l in range(self.L):
        zl = a[-1] @ self.W[l].T + self.b[l]; z.append(zl)
        a.append(act(zl) if l < self.L-1 else output_activation(zl))
    return a, z
def backward(self, a, z, Y):                # delta[L] from the loss, then delta[l] = (delta[l+1] W[l+1])
* f'(z[l])
    n = len(Y); delta = [None]*(self.L+1)
    delta[self.L] = (a[-1]-Y) if self.loss == "ce" else (a[-1]-Y)*a[-1]*(1-a[-1])
    for l in range(self.L-1, 0, -1):
        delta[l] = (delta[l+1] @ self.W[l]) * dact(a[l], z[l])
    gW = [delta[l+1].T @ a[l] / n for l in range(self.L)]
    gb = [delta[l+1].sum(0) / n for l in range(self.L)]
    return gW, gb, delta
```

(a) Gradient check

Analytic gradients are compared with central differences ($h = 10^{-5}$) for **every** weight and bias; relative error = $|g_a - g_n| \,/\, \max(|g_a| + |g_n|, 10^{-12})$.

Network	Maximum relative error
sigmoid-MSE 4-5-3-2	8.97e-9
tanh-MSE 4-5-3-2	6.35e-9
sigmoid-softmax-CE 4-5-3	6.24e-9
tanh-softmax-CE 4-6-5-3	7.07e-9

All maxima are below 10^{-8} (the threshold usually accepted is 10^{-6} to 10^{-7}), i.e. the back-propagation recurrence is implemented correctly for both loss/output pairs and for sigmoid and tanh hidden layers (ReLU is excluded from the check because its kink makes finite differences unreliable at 0).

(b) XOR: perceptron versus network

Model	Result
Single-layer perceptron, 1000 epochs, 20 seeds	best accuracy ever reached = 0.75 (3 of 4 points); final accuracies 0.25 or 0.50
MLP 2-4-1, sigmoid, squared error, η = 2, 5000 epochs, 20 seeds	accuracy 1.00 in 20 / 20 seeds; seed 0 outputs for (0,0),(0,1),(1,0),(1,1): 0.024, 0.974, 0.980, 0.024; final loss 2.86e-4

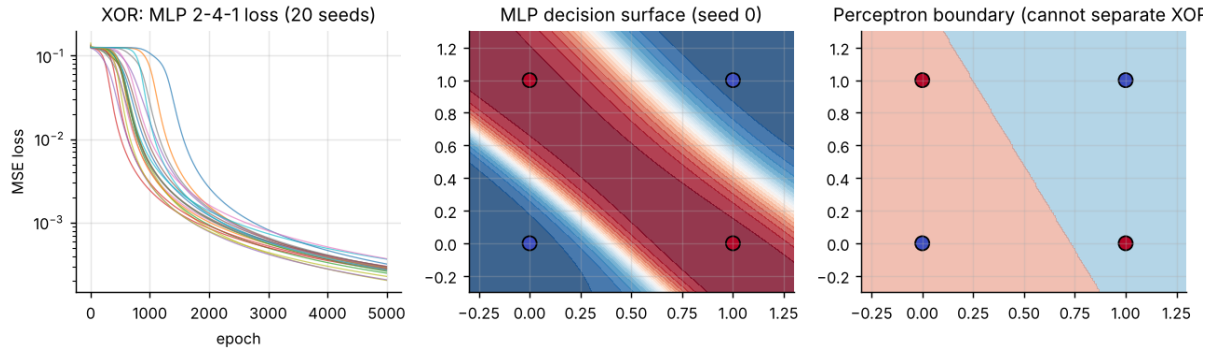


Figure 4.1 – Left: XOR loss of 20 networks. Middle: decision surface of the network. Right: the perceptron's single line can never separate XOR.

A single linear threshold unit can classify at most three of the four XOR points (the four points are not linearly separable); one hidden layer lets the network build two half-planes and combine them.

(c) Reproduction of the Task 4.4 trace in code

The NumPy network, initialised with the same weights, gives $\hat{y} = 0.56429$, $E = 0.09492$, $\delta_o = -0.10713$, $\delta = (-0.01511, 0.00524)$, updated loss 0.08514. The maximum absolute difference to the scalar hand trace over all of these quantities and all updated weights is **0.0e+0**.

Task 4.6 – Stress the design [Implement]

Data: scikit-learn 8 × 8 digits (1797 images, 64 inputs scaled to [0, 1], 10 classes), 75 / 25 stratified split (seed 42). **Model:** 64-h-10, softmax + cross-entropy, mini-batch 64, 100 epochs, same initial seed for all 32 settings. Grid: hidden ∈ {2, 8, 32, 64} × learning rate ∈ {0.001, 0.01, 0.1, 1.0} × activation ∈ {sigmoid, ReLU}.

Predictions made before the grid was run, and what happened

Prediction	Outcome	Verdict
P1: 2 hidden units cap accuracy below ≈ 80 % for every setting	Best 2-unit result: 0.562 (ReLU, lr 0.1); sigmoid best 0.529	Confirmed
P2: sigmoid with lr 0.001 and 0.01 under-fits (< 70 %) for all widths	lr 0.001: 0.10–0.26 ✓. lr 0.01: 0.20 / 0.50 / 0.78 / 0.82 for widths 2/8/32/64	Half right – wide nets reached 0.78–0.82 at lr 0.01

Prediction	Outcome	Verdict
P3: ReLU with lr 1.0 is unstable for wide layers, sigmoid lr 1.0 stable	ReLU lr 1.0 collapsed (10 % accuracy) for widths 2, 8 and 32, but the widest (64) survived and became the best model; sigmoid lr 1.0 stable	Wrong in detail – instability hit the narrower layers, not the wider
P4: best = width 32–64, lr 0.1	Best = ReLU 64, lr 1.0 (0.978); ReLU 64 lr 0.1 gives 0.971, sigmoid 64 lr 0.1 0.953	Width right, learning rate wrong (sigmoid needs the larger step)
P5: deep sigmoid net shows > 10× gradient shrinkage per two layers	see below: ≈ 20× per two layers	Confirmed

The two wrong predictions are informative: I had assumed a larger network is more fragile at a high learning rate; in fact a wider ReLU layer is more likely to keep *some* units alive after a destructive step, whereas narrow layers lose all of them (diagnosed below).

Results for all 32 settings (test set, 450 images)

Act.	Hidden	lr	Final train loss	Train acc.	Test acc.	Macro-F1
sigmoid	2	0.001	2.3038	0.101	0.102	0.019
sigmoid	2	0.01	2.1341	0.203	0.196	0.118
sigmoid	2	0.1	1.3660	0.468	0.478	0.403
sigmoid	2	1	1.0302	0.565	0.529	0.461
sigmoid	8	0.001	2.2930	0.114	0.104	0.036
sigmoid	8	0.01	2.0309	0.542	0.504	0.496
sigmoid	8	0.1	0.3924	0.926	0.900	0.899
sigmoid	8	1	0.0562	0.993	0.951	0.950
sigmoid	32	0.001	2.2765	0.174	0.182	0.143
sigmoid	32	0.01	1.8639	0.811	0.784	0.749
sigmoid	32	0.1	0.1967	0.961	0.953	0.953
sigmoid	32	1	0.0171	1.000	0.969	0.969
sigmoid	64	0.001	2.2501	0.283	0.258	0.226
sigmoid	64	0.01	1.6830	0.845	0.816	0.800
sigmoid	64	0.1	0.1782	0.962	0.953	0.953
sigmoid	64	1	0.0163	1.000	0.976	0.976
relu	2	0.001	2.1526	0.192	0.184	0.074
relu	2	0.01	1.8790	0.301	0.320	0.208
relu	2	0.1	1.0823	0.569	0.562	0.529
relu	2	1	2.3118	0.099	0.100	0.018
relu	8	0.001	2.0121	0.233	0.227	0.174
relu	8	0.01	0.6988	0.824	0.784	0.779
relu	8	0.1	0.1011	0.967	0.942	0.942
relu	8	1	2.3118	0.099	0.100	0.018
relu	32	0.001	1.9791	0.461	0.496	0.458
relu	32	0.01	0.3260	0.935	0.922	0.920
relu	32	0.1	0.0419	0.996	0.967	0.967
relu	32	1	2.3118	0.099	0.100	0.018
relu	64	0.001	1.6779	0.686	0.644	0.620
relu	64	0.01	0.2744	0.944	0.940	0.940
relu	64	0.1	0.0369	0.997	0.971	0.971
relu	64	1	0.0044	1.000	0.978	0.977

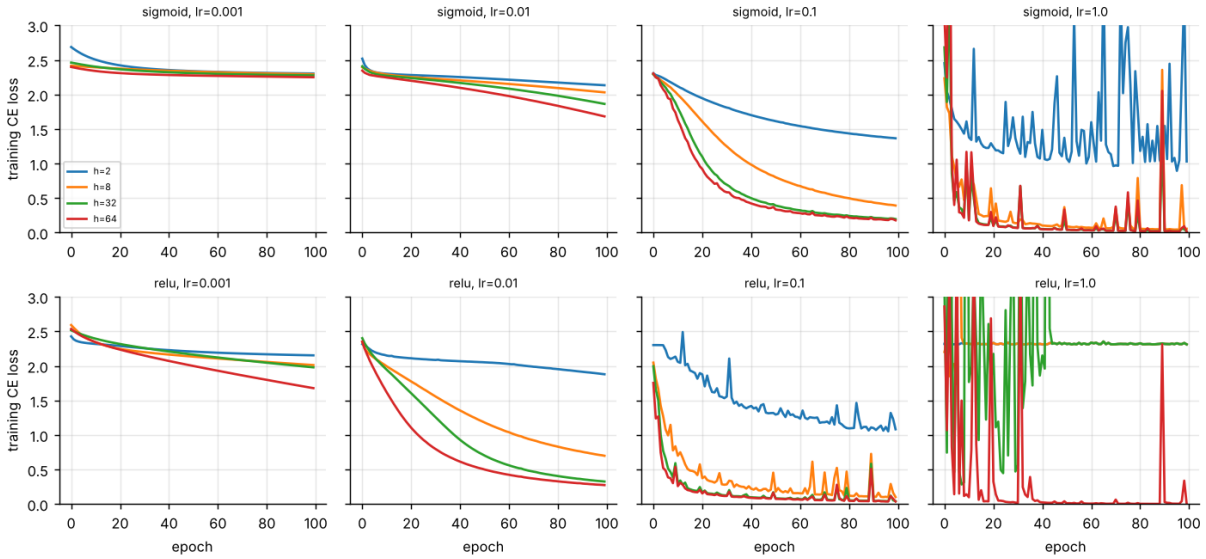


Figure 4.2 – Training-loss curves (rows: sigmoid, ReLU; columns: learning rate; lines: hidden units).

Best configuration: relu, 64 hidden units, lr 1: test accuracy 0.978, macro-F1 0.977. Its confusion matrix (rows = true digit 0–9, columns = predicted):

	0	1	2	3	4	5	6	7	8	9
0	44	0	0	0	1	0	0	0	0	0
1	0	45	0	0	0	0	0	0	1	0
2	0	0	44	0	0	0	0	0	0	0
3	0	0	0	46	0	0	0	0	0	0
4	0	0	0	0	45	0	0	0	0	0
5	0	0	0	0	0	45	0	0	0	1
6	0	0	0	0	0	0	44	0	1	0
7	0	0	0	0	0	0	0	45	0	0
8	0	3	0	0	0	1	0	1	38	0
9	0	0	0	0	0	0	0	1	0	44

Provoked failure 1: ReLU, 32 hidden units, lr = 1.0 (dead units)

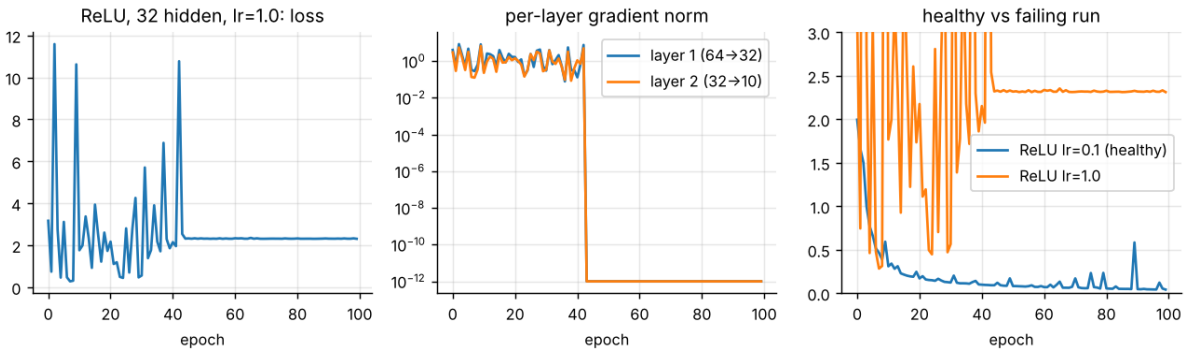


Figure 4.3 – Left: loss spikes then settles at ≈ 2.31 = ln 10 (chance). Middle: per-layer gradient norms fall to exactly 0. Right: healthy run (lr 0.1) for comparison.

> **Diagnosis.** Loss: 3.17 → 0.74 → 11.59 → 2.76 → 0.46 (epochs 1–5; peak 11.6 at epoch 3), then 2.312 ≈ ln 10 = 2.303 – the loss of a classifier that outputs the same distribution for every image. Per-layer gradient norms ($\|\partial E/\partial W\|$) were (4.12, 3.02) in epoch 1, (8.58, 5.57) in epoch 3 – large, because a step of 1.0 times these gradients overshoots – and **exactly (0, 0)** in the last epochs. At the end 100% of the hidden units output 0 for every training image; the model predicts class 0 for 447 of 450 test images (test accuracy 0.10).

Mechanism: one oversized update pushed the pre-activations of all hidden units negative; ReLU′(z) = 0 there, so $\delta^{(1)} = 0$, so no gradient flows to the hidden weights – the units can never recover ("dying ReLU"). This is not numerical divergence to NaN but a permanent collapse, and only the gradient norms (not the loss alone) show that the network is dead, not merely slow.

Provoked failure 2: vanishing gradient in a deep sigmoid network

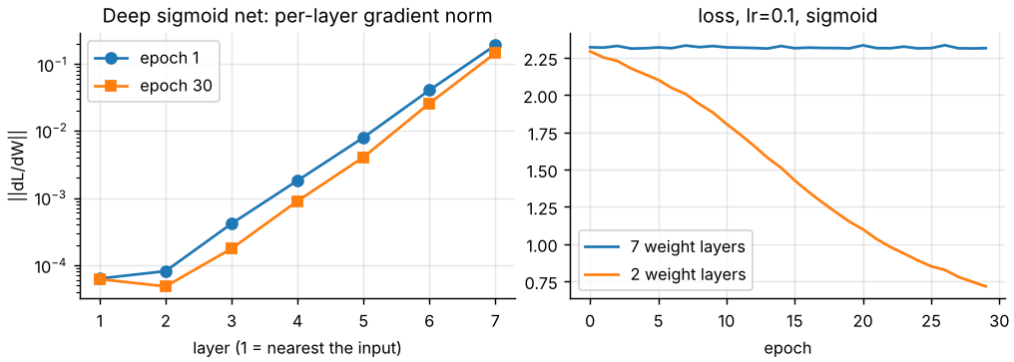


Figure 4.4 – Left: gradient norm per layer (layer 1 is next to the input). Right: loss of a 7-weight-layer sigmoid network versus a 2-layer one.

Diagnosis. A 64-32-32-32-32-32-32-10 sigmoid network (lr 0.1, 30 epochs) has per-layer gradient norms 6.3e-5, 8.1e-5, 4.2e-4, 1.8e-3, 8.0e-3, 4.1e-2, 1.9e-1 from the input side to the output side: each layer closer to the input gets roughly 4–5× smaller gradients (≈ 20× per two layers, ≈ 3000× over the network) because every backward step multiplies by σ′(z) ≤ 0.25. After 30 epochs its loss is 2.314 (accuracy 0.10 = chance) while the shallow sigmoid network with identical settings reaches loss 0.715 and accuracy 0.864. Remedies suggested by the diagnosis: ReLU-family activations with He initialisation, normalisation layers, residual connections or a larger learning rate for early layers.

TASK 5 – Same Dataset, Different Learning Reasoning (8 marks)

Common data set. Labelled substation states: two sensed quantities (load ratio and temperature deviation, both standardised) and a label 1 = *hazardous overload*, 0 = normal. Hazardous states lie in an outer ring around the normal operating region, i.e. the classes are **not linearly separable** (generated with `make_circles`, 1200 states, noise 0.18, 50 % hazardous; hazardous = outer ring). Split 70 % / 30 % stratified (840 train / 360 test, seed 42); the same split is used by every model in 5.1 and 5.2.

Task 5.1 – Statistical learning with complete data: Naive Bayes

[Design] Pseudocode with maximum-likelihood counts and Laplace smoothing

Pseudocode

```
ALGORITHM NaiveBayesTrain(D, α)
INPUT : D = {(x_i, y_i)}, x_i = (x_i1 … x_id) with x_ij ∈ {1..V_j};  classes C;  smoothing α ≥ 0
OUTPUT: prior[c], θ[c][j][v]
BEGIN
 1  FOR EACH class c IN C DO  N_c ← number of examples with label c;  prior[c] ← N_c / n  END FOR
 2  FOR EACH c, j, v DO  count[c][j][v] ← number of examples with label c and x_ij = v  END FOR
 3  FOR EACH c, j, v DO  θ[c][j][v] ← (count[c][j][v] + α) / (N_c + α · V_j)  END FOR   // α = 0:
pure ML
 4  RETURN prior, θ
END

ALGORITHM NaiveBayesPredict(x)
 1  FOR EACH class c DO  s[c] ← log prior[c] + SUM over j of log θ[c][j][x_j]  END FOR   // log 0
= −∞
 2  IF s[c] = −∞ for every c THEN RETURN "undefined" (fall back to the prior)
 3  m ← MAX over c of s[c];   P[c] ← exp(s[c] − m) / SUM over c′ of exp(s[c′] − m)
 4  RETURN argmax over c of s[c], together with P
END
```

Toy table – why a zero count breaks classification. Three training examples:

#	Load	Temp	Class
1	High	Hot	Hazard
2	High	Mild	Hazard
3	Low	Mild	Normal

Query (Load = Low, Temp = Hot). Priors: P(Hazard) = 2/3, P(Normal) = 1/3. **α = 0:** P(Low | Hazard) = 0/2 = 0 and P(Hot | Normal) = 0/1 = 0, so score(Hazard) = 2/3 · 0 · ½ = 0 **and** score(Normal) = 1/3 · 1 · 0 = 0. Both classes have score 0; the posterior 0/0 is undefined – one unseen feature value vetoes a class completely, whatever the other features say, and here it vetoes both. **α = 1** (V_{Load} = V_{Temp} = 2): P(Low|Hazard) = 1/4, P(Hot|Hazard) = 2/4, P(Low|Normal) = 2/3, P(Hot|Normal) = 1/3, giving scores 0.0833 and 0.0741, i.e. posterior P(Hazard) = 0.529 – a defined, sensible, low-confidence answer.

[Implement] From-scratch categorical Naive Bayes, α ∈ {0, 0.1, 1, 10}

The two continuous features are discretised into 12 quantile bins (edges from the training set only). Log-loss uses probabilities clipped to $[10^{-15}, 1-10^{-15}]$ (needed for α = 0); 0/0 rows are given the prior and counted.

Source code (excerpt)

```
class CategoricalNB:
    def fit(self, B, y):                        # B: bin indices (n, d)
        self.prior = np.array([(y == c).mean() for c in self.classes])
        self.count[ci, j] = np.bincount(B[y == c, j], minlength=n_bins)   # ML counts
    def log_joint(self, B):
        for ci in range(len(self.classes)):
            theta = (self.count[ci] + alpha) / (self.nc[ci] + alpha * n_bins)    # Laplace smoothing
            for j in range(self.d): lp[:, ci] += np.log(theta[j, B[:, j]])       # log 0 = -inf allowed
```

(i) Full training set (840 rows):

α	Accuracy	Precision	Recall	F1	Log-loss	Zero-count cells	Undefined test rows
0	0.906	0.910	0.900	0.905	0.266	2	0
0.1	0.906	0.910	0.900	0.905	0.267	2	0
1	0.906	0.910	0.900	0.905	0.277	2	0
10	0.906	0.910	0.900	0.905	0.346	2	0

With 840 rows and 12 bins only 2 of 48 (class, feature, bin) cells are empty, so accuracy, precision, recall and F1 are identical for all α; only the **log-loss** reacts: it grows from 0.266 (α = 0) to 0.346 (α = 10) because strong smoothing flattens the probabilities (under-confidence).

(ii) Scarce data – 40 training rows, 20 random subsets, same 360 test rows (this is where smoothing matters):

α	Accuracy (mean ± sd)	F1	Log-loss (mean ± sd)	Test rows with 0/0 (of 360)
0	0.720 ± 0.071	0.706	4.119 ± 1.481	66.7
0.1	0.760 ± 0.053	0.763	0.611 ± 0.117	0.0
1	0.765 ± 0.045	0.765	0.506 ± 0.045	0.0
10	0.659 ± 0.091	0.625	0.631 ± 0.016	0.0

> **Interpretation.** With α = 0 on scarce data **67 of 360 test rows (19%) are undefined** and the log-loss explodes to 4.12; any small positive α removes the failure (log-loss ≈ 0.5–0.6). The best accuracy/F1 is at α = 0.1–1 (≈ 0.76); α = 10 over-smooths (accuracy 0.659, below α = 0). Smoothing is therefore a bias–variance dial: too little → zero-count vetoes, too much → the data are ignored. **Boundary test:** α = 0 on the full data still works only because zero cells are rare.

Task 5.2 – Change the assumption: kernel machines

[Design] Which assumption became unsafe?

The earlier models make structural assumptions that the ring-shaped hazard region violates: (1) **linear separability** – perceptron, logistic regression and a linear SVM can only draw one straight line, which cannot enclose a region; (2) Naive Bayes' **conditional independence** of the features given the class (load and temperature deviation are *jointly* constrained to lie at a distance from the centre, so they are dependent within a class) – NB survives only because quantile bins make its per-feature densities non-linear; (3) the tree's axis-aligned boxes approximate a circle only with many splits. The assumption that really breaks for the linear learners is **"a hyperplane in the input space separates the classes".**

- **Margin.** Among all separating hyperplanes the SVM picks the one with the largest distance to the nearest training points (margin $2/\|w\|$); maximising it is a regularised, well-posed problem and controls capacity. The soft-margin parameter C trades margin width against training errors.
- **Support vectors.** Only the points on or inside the margin (and the misclassified ones) have non-zero dual weights α_i; the decision function $f(x) = \Sigma_i\, \alpha_i\, y_i\, K(x_i, x) + b$ depends on them alone, so the model size = the number of support vectors.
- **Kernel function.** $K(x, x') = \phi(x)\cdot\phi(x')$ is an inner product in a (possibly infinite-dimensional) feature space reached without computing ϕ. The RBF kernel $K = \exp(-\gamma\|x - x'\|^2)$ lets a hyperplane in that space become a closed curve in the input space; γ sets the radius of influence (large γ → wiggly, over-fit; small γ → nearly linear).

[Implement] Linear vs RBF SVM, 5-fold CV for C (and γ); comparison with NB and the Task-2 tree

Grid: linear $C \in \{0.01, 0.1, 1, 10, 100\}$; RBF $C \in \{0.1, 1, 10, 100\} \times \gamma \in \{0.01, 0.1, 1, 10\}$; 5-fold stratified CV on the training set, score = F1 of the hazardous class; the best setting is refitted on all training data. The

tree is my Task-2 learner with depth chosen by the same 5-fold CV; Naive Bayes uses α = 1. Fit time is the average of repeated fits (small data – only the order of magnitude is meaningful).

Model	Best settings	Support vectors	Accuracy	F1	Recall	Fit time (ms)
Linear SVM	C = 10	836	0.506	0.447	0.400	16.24
RBF SVM	C = 10, γ = 0.1	239	0.947	0.946	0.917	5.39
Naive Bayes (alpha=1)	α = 1, 12 bins	–	0.906	0.905	0.900	0.08
Decision tree (Task 2, depth<=8)	depth ≤ 8, 26 leaves	–	0.897	0.892	0.850	4.47

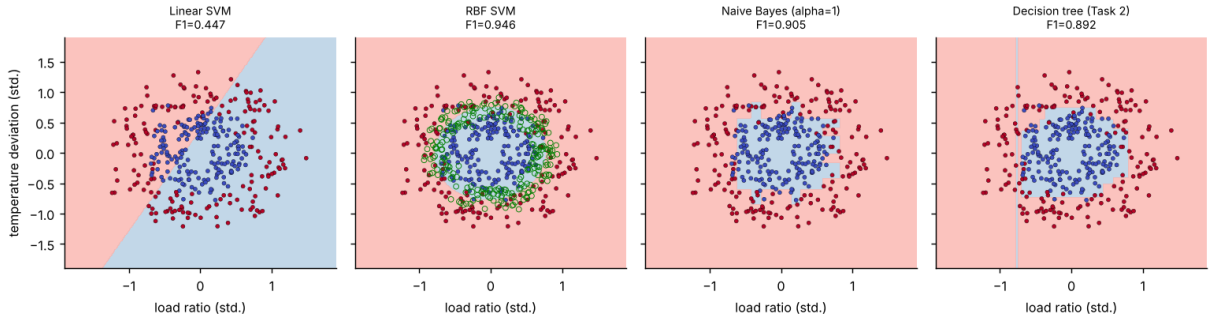


Figure 5.1 – Decision regions on the identical test set (red region = predicted hazardous). Green circles: support vectors of the RBF SVM.

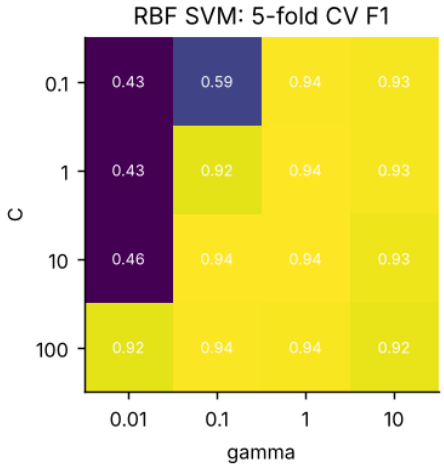


Figure 5.2 – Five-fold CV F1 over the RBF grid: with small C and γ = 0.01 the model is almost linear (F1 ≈ 0.43); large γ (10) is slightly worse (0.92–0.93) than the plateau of 0.94.

Interpretation. The linear SVM is at chance (accuracy 0.506) and needs **836 of 840 points as support vectors** – nearly the whole training set lies inside its useless margin. The RBF SVM reaches accuracy 0.947 / F1 0.946 with 239 support vectors (28 % of the data), which trace the boundary ring in Figure 5.1. Naive Bayes (0.906) and the tree (0.897) are far better than linear but produce boxy, axis-aligned regions and trail the RBF SVM by 4–5 accuracy points; the RBF-SVM advantage over NB is about 14 test states out of 360, so with one split it is suggestive rather than proven. Costs: RBF training is slower than NB by orders of magnitude and the model must keep its support vectors; the tree and NB are cheaper and interpretable. **Boundary/failure test:** γ = 0.01 with C ≤ 10 under-fits to near-linear behaviour (CV F1 0.43–0.47) and γ = 10 is slightly worse (0.924–0.933) than the 0.939 plateau, so the result is robust over a wide region of the grid; the linear model's F1 of 0.447 is misleading – it comes from predicting many states hazardous (recall 0.400, precision 0.507), which shows why accuracy and F1 must be read together.

Task 5.3 – Change the requirement: explanation-based learning

[Design] Domain theory (10 first-order Horn rules)

Variables are lower-case (s = substation, t = transformer, f = feeder); *operational* predicates are directly sensed or recorded and may appear in a learned rule; the others are intermediate concepts.

Rule	Definition
R1	Hazardous(s) ⇐ Overload(s) ∧ ThermalStress(s)
R2	Hazardous(s) ⇐ ProtectionFailed(s) ∧ FaultCurrentHigh(s)
R3	Overload(s) ⇐ Transformer(s,t) ∧ HighLoad(t)
R4	ThermalStress(s) ⇐ Transformer(s,t) ∧ HotOil(t)
R5	ThermalStress(s) ⇐ Transformer(s,t) ∧ AmbientHigh(s) ∧ CoolingDegraded(t)
R6	CoolingDegraded(t) ⇐ FanFailed(t)
R7	CoolingDegraded(t) ⇐ RadiatorBlocked(t)
R8	ProtectionFailed(s) ⇐ RelayStale(s)
R9	ProtectionFailed(s) ⇐ BreakerStuck(s)
R10	FaultCurrentHigh(s) ⇐ Feeder(s,f) ∧ ShortCircuitSeen(f)

Operational predicates: Transformer, HighLoad, HotOil, AmbientHigh, FanFailed, RadiatorBlocked, RelayStale, BreakerStuck, Feeder, ShortCircuitSeen.

The single explained example. Facts: Transformer(S1,T1), HighLoad(T1), AmbientHigh(S1), FanFailed(T1), Feeder(S1,L1). Goal: Hazardous(S1). **Explanation tree** produced by the prover:

Explanation (proof) tree

```
Hazardous(S1)   [by R1]
    Overload(S1)   [by R3]
        Transformer(S1, T1)   [operational fact]
        HighLoad(T1)   [operational fact]
    ThermalStress(S1)   [by R5]
        Transformer(S1, T1)   [operational fact]
        AmbientHigh(S1)   [operational fact]
        CoolingDegraded(T1)   [by R6]
            FanFailed(T1)   [operational fact]
```

Generalisation (EBG). The proof *structure* is kept and the constants are replaced by variables by unifying each rule head with the generalised sub-goal, starting from Hazardous(?x): R1 introduces Overload(?s) and ThermalStress(?s); R3 contributes Transformer(?s,?t) ∧ HighLoad(?t); R5 contributes Transformer(?s,?u) ∧ AmbientHigh(?s) ∧ CoolingDegraded(?u); R6 turns CoolingDegraded(?u) into FanFailed(?u). Collecting the operational leaves gives the learned rule:

> **Hazardous(s) ⇐ Transformer(s,t) ∧ HighLoad(t) ∧ Transformer(s,u) ∧ AmbientHigh(s) ∧ FanFailed(u)**

Note what was *dropped* (Feeder(S1,L1) – not used in the proof) and what was *generalised* (S1 → s; T1 → two variables t and u, because the proof never forced the loaded transformer and the fan-failed transformer to be the same).

[Implement] 20 designed test cases, inference-step comparison

A small SLD-resolution prover (unification, renaming apart, backtracking) was written from scratch. “Inference steps” = number of goal literals the engine attempts to resolve (successes and failures). Each case is a substation with its own facts; the *theory* column is the result of proving Hazardous(s) from the domain theory (ground truth), the *learned* column is the result of the learned rule alone.

#	Facts (besides Transformer(S,T))	Theory	Learned	Steps theory	Steps learned
1	HighLoad(T), AmbientHigh(S), FanFailed(T)	hazard	hazard	15	6
2	HighLoad(T), HotOil(T), AmbientHigh(S), FanFailed(T)	hazard	hazard	15	6
3	HighLoad(T), AmbientHigh(S), FanFailed(T), RelayStale(S)	hazard	hazard	17	6
4	HighLoad(T), AmbientHigh(S), FanFailed(T), RadiatorBlocked(T)	hazard	hazard	15	6

#	Facts (besides Transformer(S,T))	Theory	Learned	Steps theory	Steps learned
5	HighLoad(T), AmbientHigh(S), RadiatorBlocked(T)	hazard	safe	15	6
6	HighLoad(T), HotOil(T)	hazard	safe	12	5
7	HighLoad(T), HotOil(T), AmbientHigh(S)	hazard	safe	15	6
8	RelayStale(S), Feeder(S, L), ShortCircuitSeen(L)	hazard	safe	10	3
9	BreakerStuck(S), Feeder(S, L), ShortCircuitSeen(L)	hazard	safe	10	3
10	RelayStale(S)	safe	safe	9	3
11	Feeder(S, L), ShortCircuitSeen(L)	safe	safe	7	3
12	HighLoad(T)	safe	safe	12	5
13	AmbientHigh(S), FanFailed(T)	safe	safe	7	3
14	–	safe	safe	7	3
15	HotOil(T)	safe	safe	7	3
16	HighLoad(T), AmbientHigh(S)	safe	safe	15	6
17	AmbientHigh(S), FanFailed(T), RadiatorBlocked(T)	safe	safe	7	3
18	HighLoad(T), AmbientHigh(S), Transformer(S, T), FanFailed(T)	hazard	hazard	21	9
19	AmbientHigh(S), Transformer(S, T), HighLoad(T), FanFailed(T)	hazard	hazard	21	9
20	HighLoad(T), AmbientHigh(S), FanFailed(T), BreakerStuck(S), Feeder(S, L), ShortCircuitSeen(L)	hazard	hazard	18	6

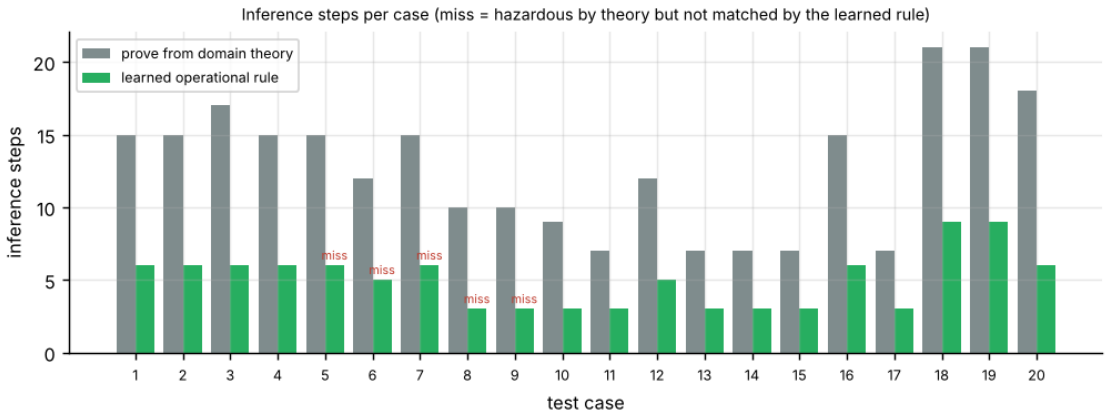


Figure 5.3 – Inference steps per test case; "miss" marks hazardous cases the learned rule does not recognise.

> **Results.** Agreement with the theory on 15 of 20 cases; 12 cases are hazardous by theory and the learned rule recognises 7 of them (58%) with **no false alarms** (sound but incomplete). On the covered cases the learned rule needs on average 6.9 steps against 17.4 for the theory – **2.5× faster**. The 5 missed cases are hazardous through other proof paths (hot oil – cases 6, 7; blocked radiator – case 5; protection failure – cases 8, 9) that the single example did not exercise. Using "learned rule first, theory as fall-back" costs 11.7 steps per case versus 12.8 for the theory alone – only a 9% saving, because every miss pays for both; the speed-up grows with the rule's coverage.

What was learned that the domain theory did not already entail?

- **Logically nothing.** The learned rule is a deductive consequence of the domain theory (it is a compressed proof), so it can never classify a case differently from the theory – confirmed by 0 false alarms. EBL does not add knowledge; it adds **operationality and speed**: a one-step rule over directly sensed facts instead of a 10–21 step search, and it generalises from one example to every substation and every pair of transformers (cases 18–20 are different from S1/T1 yet recognised).
- **A side effect worth noticing:** because t and u are independent variables, case 18 (one transformer heavily loaded, a *different* transformer with a failed fan, hot ambient) is classified hazardous. That is what the theory says, but it is probably physically wrong – the learned rule exposed a modelling gap (R3 and R5 should probably refer to the same transformer). Inspecting learned rules is a way of debugging the theory.

- **Limitation:** the rule covers one explanation path; learning the other paths needs further explained examples, and the rule set grows with the number of distinct proofs (utility problem).

TASK 6 – Design Challenge: Reinforcement Learning for an Energy-Aware Smart Controller (13 marks)

Stage 1 – Interpret the problem (no algorithm chosen yet)

Aspect	Interpretation
Input (what the controller observes)	Every 15 minutes: time of day, indoor temperature, whether the hall is occupied now, outdoor temperature (and, through the time slot, the electricity price). It does not observe the exact head-count, the future weather or the future occupancy.
Output (action)	One of {Off, Low, High} applied for the next 15 minutes (cooling power 0 / 3 / 6 kW).
Objective	Minimise electricity cost over the day while keeping the room comfortable (22–26 °C) whenever it is occupied: a sequential trade-off between money and comfort.
Constraints	Only three discrete actions; the room cannot be cooled instantly (thermal inertia – cooling now affects the temperature one hour later); price changes by time slot (cheap at night, peak 14–18 h); lectures start abruptly at 09:00 and 14:00; occupancy is uncertain (cancelled classes, 40–100 students); comfort is only required when occupied.
Decisions to make	When to start cooling before a lecture (pre-cool), whether to use Low or High, when to stop, whether to shift cooling to cheap hours, how much discomfort is ever worth the saving.
Episode	Begins at 00:00 with a new random day (outdoor temperature profile, lecture timetable with cancellations) and indoor temperature ≈ N(28, 1) °C. Ends after 96 steps at 24:00 (a day), a terminal state with no future value; time of day is part of the state, so this finite-horizon cut-off does not violate the Markov property.

Stage 2 – Propose the strategy

Alternative	What it needs / assumes	Fit to this problem
Fixed-rule thermostat	A hand-written rule (e.g. on 08–18 h, High if T > 26, Low if T > 24)	No learning, no use of price or arrival time; cannot pre-cool or shift load. Kept as the baseline.
Supervised imitation of past operators	A log of expert (state, action) pairs	Copies, and is capped by, the operators' skill; labels do not contain the *cost* of an action, so it cannot discover cheaper behaviour; no data exist for new tariffs.
Model-based (dynamic programming)	Known transition probabilities and rewards	Optimal if the model is right, but occupancy cancellations and weather are random and not known in closed form; the model would have to be estimated first and its errors propagate.
SARSA (on-policy TD)	Experience from interaction; learns the value of the policy actually followed (incl. exploration)	Suitable, and safer when exploring on the real building because exploration risk is priced in; learns the value of the exploring policy, not of the greedy one.
Q-learning (off-policy TD)	Experience from interaction or from a simulator / logged data	Chosen. See reasons below.

Why Q-learning, argued from the problem's characteristics:

- **Sequential, delayed consequences.** Cooling now pays off 1–2 hours later (thermal inertia) and the price changes over the day. That is credit assignment over time, which rules out supervised learning on immediate labels and bandit-style learning; temporal-difference learning propagates future costs back through the discounted return.
- **Unknown, stochastic dynamics.** Occupancy and weather are random and not given, so a model-free method that learns from sampled transitions is appropriate; no model has to be specified.
- **Small discrete state and action spaces** (1008 states × 3 actions, below). A table is exact, interpretable (each entry is a value in ₹), and Q-learning is guaranteed to converge for tabular representations when all pairs are visited and the step size is decreased/small; no function approximator is needed.

- **Off-policy learning.** It learns the value of the *greedy* policy while behaving exploratorily, so the same learner can be trained on simulator data, on logged data from the old thermostat, and on a new tariff without changing the target – useful here because we want the cost-optimal policy, not the value of a noisy one. The price: no protection from risky exploration on the real building, which is why exploration is done in a simulator first.
- **Reward is a scalar mix of two units (₹ and °C).** RL optimises any such return directly; the trade-off is set by a weight, whose danger is examined in the attack in Stage 6.

SARSA is a serious alternative; as a post-hoc sanity check (not the basis of the choice) 10 SARSA seeds reached a mean reward of -392.8 ± 12.4, statistically indistinguishable from Q-learning (-405.8 ± 25.2); on this problem the choice therefore matters less than the reward design.

Stage 3 – Identify the critical information

Element	Definition	Why
State s = (hour, T-bin, occ, out-band)	hour ∈ {0..23}; indoor-temperature bin ∈ 7 bins with edges 22, 23.5, 25, 26, 27, 29 °C; occ ∈ {0 (empty), 1 (occupied)}; outdoor band ∈ {<30, 30–34, >34 °C}. 24 × 7 × 2 × 3 = 1008 states.	See Markov argument below.
Actions	a ∈ {Off = 0 kW, Low = 3 kW (−0.6 °C/step), High = 6 kW (−1.2 °C/step)}	Given by the problem.
Reward	r = − price(hour) · kWh(a) − [λ_o · 1(violation) + λ · deviation]; kWh = kW × 0.25 h; violation = occupied and T outside [22, 26] °C after the step; deviation = °C outside the band; λ_o = 5 ₹, λ = 15 ₹/°C. Prices (₹/kWh): 0–6 h: 5; 6–9: 8; 9–14: 9; 14–18: 12; 18–22: 9; 22–24: 5.	Energy cost enters in rupees; discomfort is converted to rupees with weights chosen so that a 1 °C violation (≈ ₹20) costs about as much as one peak-hour High step (₹18).
Exploration	ε-greedy; ε falls linearly from 1.0 to 0.05 over the first 3000 of 6000 episodes, then stays 0.05; Q initialised to 0 (optimistic, because every reward ≤ 0, which drives early exploration).	Visit all 1008 × 3 pairs early, then exploit.
Discount γ	0.97 per 15 min (effective horizon ≈ 33 steps ≈ 8 h)	Pre-cooling at 07:00 for a 09:00 lecture (8 steps) and night cooling for the next day's start must still be worth their cost; γ = 0.97 retains 78 % of the value after 8 steps.
Learning rate	α = 0.1, constant	Noisy transitions, non-stationary targets early on.

Why this state is Markov enough. The hour fixes the lecture timetable, the price band and the diurnal outdoor trend; the temperature bin is the integral of all past heating and cooling (thermal inertia means history matters only through T); the occupancy flag fixes the heat load; the outdoor band fixes the drift. What is left out (exact head-count, noise in the weather, whether a lecture was cancelled) is modelled as transition noise. It is not perfectly Markov – the bin coarsening hides ±0.7 °C – but the transition and reward depend on the *past* only through the chosen state to a good approximation, which is all tabular TD needs.

Stage 4 – Complete pseudocode

SIMATS-style pseudocode – complete

```
ALGORITHM LearnACPolicy(Simulator, γ, α, ε0, εmin, E)
INPUT : Simulator (see below); γ = 0.97; α = 0.1; ε0 = 1.0; εmin = 0.05; E = 6000 episodes
        S = {(hour, Tbin, occ, band)}, |S| = 1008; A = {0 = Off, 1 = Low, 2 = High}
OUTPUT: policy π[s] ∈ A for every s ∈ S;  action-value table Q
BEGIN
 1  FOR EACH s ∈ S, a ∈ A DO  Q[s][a] ← 0  END FOR
 2  FOR episode ← 1 TO E DO
 3      ε ← MAX(εmin, ε0 − (ε0 − εmin) · episode / (0.5 · E))        // linear decay, then
constant
 4      s ← Simulator.Reset()                                          // 00:00, new random day, T
~ N(28,1)
 5      FOR step ← 1 TO 96 DO
 6          u ← Uniform(0, 1)
 7          IF u < ε THEN a ← a random action from A
 8          ELSE a ← argmax over a' of Q[s][a']   (ties → smallest index, i.e. Off)
 9          END IF
10          (s', r, done) ← Simulator.Step(a)
```

```
11          IF done THEN target ← r                                    // terminal: no future value
12          ELSE target ← r + γ · MAX over a′ of Q[s′][a′]
13          END IF
14          Q[s][a] ← Q[s][a] + α · (target − Q[s][a])                 // temporal-difference
update
15          s ← s′
16          IF done THEN EXIT inner loop
17      END FOR
18  END FOR                                                            // termination: E episodes
completed
19  FOR EACH s ∈ S DO  π[s] ← argmax over a of Q[s][a]  END FOR         // policy extraction
20  RETURN π, Q
END


PROCEDURE Simulator.Reset()
    draw the day: base ~ Normal(31, 1.5); outdoor[k] ← base + 5·sin(2π(k/4 − 9)/24) + AR(1) noise
(ρ = 0.8, sd 0.25)
    FOR EACH lecture hour h IN {9, 10, 11, 12, 14, 15, 16}: n ← Uniform integer 40..100 WITH prob
0.9 ELSE 0; occupancy of the 4 steps of h ← n
    k ← 0;  T ← Normal(28, 1);  RETURN State(k, T)
PROCEDURE Simulator.Step(a)
    T ← T + 0.06 · (outdoor[k] − T) + 0.0045 · occupancy[k] − cool[a] + Normal(0, 0.05)    // cool
= (0, 0.6, 1.2)
    kWh ← power[a] · 0.25 (power = 0, 3, 6 kW);  cost ← price(hour(k)) · kWh
    k ← k + 1;  occupied ← occupancy[k] > 0 (use occupancy[k−1] on the last step)
    deviation ← MAX(0, T − 26, 22 − T) IF occupied ELSE 0;  violation ← 1 IF deviation > 0 ELSE 0
    r ← −cost − (5 · violation + 15 · deviation);  done ← (k = 96)
    RETURN State(k, T), r, done
```

Everything needed for an independent implementation is above: state discretisation (Stage 3), the transition and reward of the simulator, initialisation, action selection, update, termination and policy extraction. The state index is State(k, T) = (hour = ⌊k/4⌋, Tbin(T), occupied(k), band(outdoor[k])).

Stage 5 – Desk-check on a toy environment

Toy environment: states {Comfortable, Warm, Hot}, actions {Off, Cool}; deterministic transitions and rewards (reward = −energy − discomfort, with energy 0 / 2 and discomfort 0 / 1 / 5 for Comfortable / Warm / Hot after the step):

State	Action	Next state	Reward
Comfortable	Off	Warm	−1
Comfortable	Cool	Comfortable	−2
Warm	Off	Hot	−5
Warm	Cool	Comfortable	−2
Hot	Off	Hot	−5
Hot	Cool	Warm	−3

Q-learning with α = 0.5, γ = 0.9, all Q = 0 initially; Q(s,a) ← Q(s,a) + α [r + γ max Q(s′,·) − Q(s,a)]. The action sequence is scripted to include exploration (steps 1, 2, 4):

Update	State → action	Reward	Next state	max Q(s′,·)	Q(s,a) before → after
1	Warm → Off	-5	Hot	0.00	0.000 → -2.500
2	Hot → Cool	-3	Warm	0.00	0.000 → -1.500
3	Warm → Cool	-2	Comfortable	0.00	0.000 → -1.000
4	Comfortable → Off	-1	Warm	-1.00	0.000 → -0.950
5	Warm → Cool	-2	Comfortable	0.00	-1.000 → -1.500
6	Comfortable → Cool	-2	Comfortable	0.00	0.000 → -1.000

Worked example, update 4: (Comfortable, Off) → Warm, r = −1; max Q(Warm, ·) = max(−2.5, −1.0) = −1.0; target = −1 + 0.9·(−1.0) = −1.9; new Q = 0 + 0.5·(−1.9 − 0) = **−0.95**. Update 5 shows the value falling again (−1.0 → −1.5) because the better-informed target uses Q(Comfortable, ·) = 0 – estimates are revised as neighbouring values become known. After six updates the greedy policy is Warm → Cool (−1.5 vs −2.5 for

Off) and Comfortable → Off, Hot → Off (still untested actions, a reminder that exploration is needed). The code reproduces these numbers exactly.

Stage 6 – Implement, instrument and attack the design

Source code (excerpt)

```
def train(seed, episodes, alpha=0.1, gamma=0.97, eps0=1.0, eps_min=0.05, eps_frac=0.5, lam=15.0, lam0=5.0,
price=price_standard):
    rng = np.random.default_rng(seed); Q = np.zeros((N_STATES, N_ACTIONS))
    for ep in range(episodes):
        eps = max(eps_min, eps0 - (eps0 - eps_min) * ep / (eps_frac * episodes))
        env = Env(make_day(rng), lam, lam0, price); s = env.reset()
        for t in range(96):
            a = rng.integers(3) if rng.random() < eps else greedy(Q[s])
            s2, r, done, _ = env.step(a)
            target = r if done else r + gamma * Q[s2].max()
            Q[s, a] += alpha * (target - Q[s, a])
            s = s2
            if done: break
    return Q
```

Protocol. 20 independent training runs (seeds 0–19) of 6000 episodes each; each learned *greedy* policy is evaluated on the **same 200 unseen days** (fixed generator, never used in training) and compared with the scheduled thermostat on exactly the same days. Greedy evaluation on 30 further fixed days every 100 episodes gives the convergence measure: **episodes to convergence** = first checkpoint at which the 5-checkpoint moving average of that evaluation reward is within 5 % of the plateau (mean of the last 10 checkpoints).

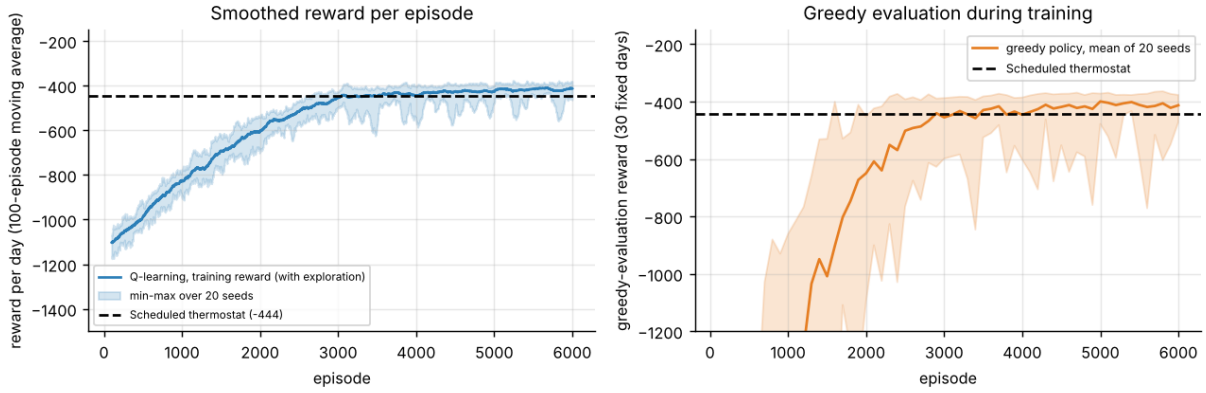


Figure 6.1 – Left: reward per episode (100-episode moving average) over 20 seeds, including exploration. Right: greedy-policy reward on fixed evaluation days. The dashed line is the fixed-rule thermostat.

Results over 20 seeds (200 unseen days each; learned = mean ± sd over seeds):

Controller	Reward / day	Energy (kWh/day)	Cost (₹/day)	Comfort-violation steps/day	Deviation (°C·steps)
Q-learning (learned)	-405.8 ± 25.2	35.14 ± 1.72	338.8 ± 19.1	4.12 ± 1.82	3.09 ± 2.03
Scheduled thermostat (fixed rule, 08–18 h)	-444.1	37.913	385.4	7.710	1.344
Always-on thermostat (24 h, for reference)	-587.5	59.648	533.2	7.115	1.249

Episodes to convergence: **3515 ± 605** (mean ± sd over seeds). The learned policy has a higher reward than the scheduled thermostat in **19 of 20 seeds** (mean improvement 8.6 % ± 5.7 %), uses less energy in 19 / 20 and has fewer violation steps in 19 / 20.

> **Honest reading of the numbers.** The improvement is real but modest (≈ 9 %) and not uniform: Figure 6.1 shows that some seeds dip below the thermostat for stretches, and one seed does not beat it at all. The learned policy has *fewer* violation steps (4.1 vs 7.7) but a *larger* total deviation (3.1 vs 1.3 °C·steps): it violates less often but, when it does, by more.

What the learned policy actually does

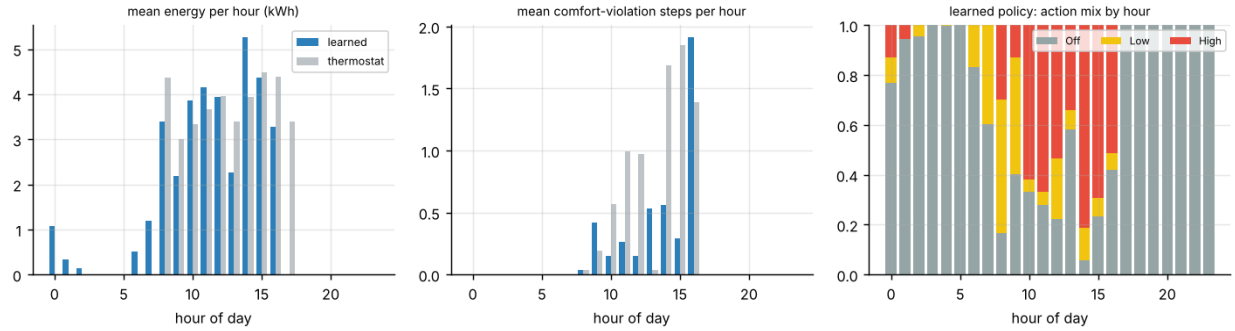


Figure 6.2 – Median-reward seed, 200 test days: energy, violation steps and action mix by hour. The learned controller uses energy before the lectures and in the afternoon, almost nothing in the evening.

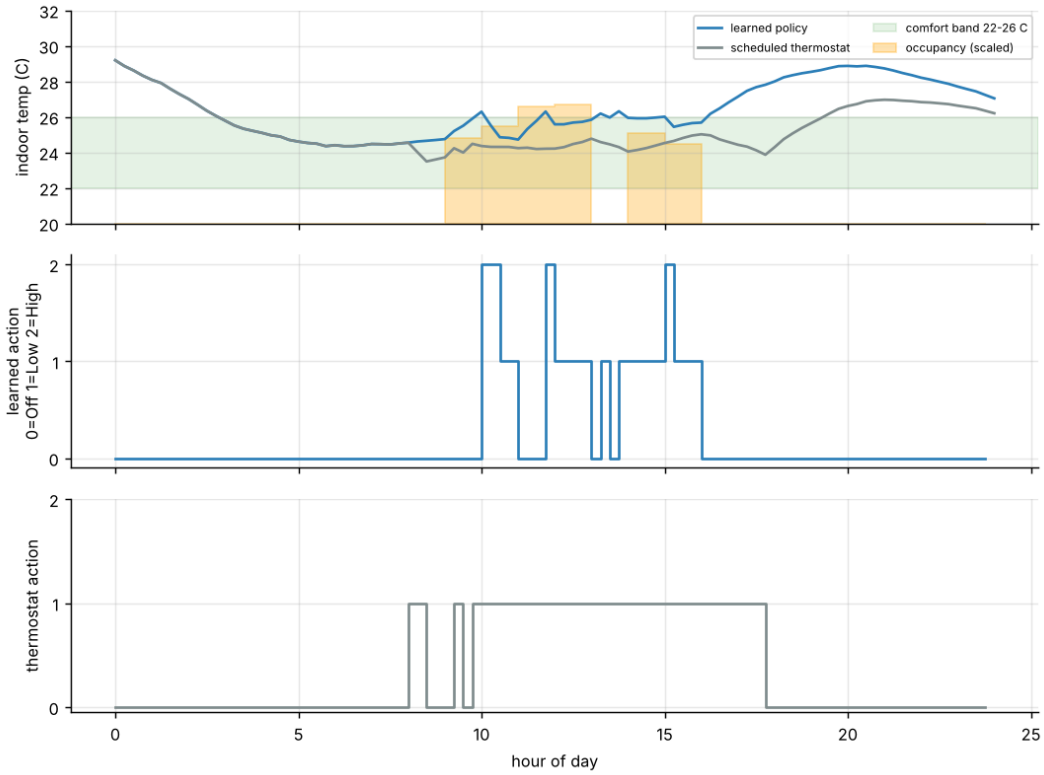


Figure 6.3 – One test day (day 4, fixed in advance, median seed 0). Learned: reward -200, 17.3 kWh, 176 ₹, 3 violation steps; thermostat: -269, 26.3 kWh, 269 ₹, 0 violations.

- **Load shifting.** It cools at night when electricity costs ₹5 (1.08 kWh in hour 0), so that the room starts the day cooler, then uses little between 06:00 and 08:00 (0.5 and 1.2 kWh per hour); most of the morning energy is used at 08:00–09:00 (3.39 kWh in hour 8). The thermostat only switches on at 08:00 and has to catch up while the lecture is running – hence its larger morning violation counts (hours 10–12: 0.56, 0.99, 0.97 vs 0.15, 0.27, 0.15 for the learned policy).
- **Predicted but not observed:** I expected clear pre-cooling in the 06:00–09:00 window; the learned policy does its pre-cooling **at night** and in the 08:00 hour, with little in 06:00–08:00 (prediction partly wrong; the cheaper night tariff made earlier shifting more attractive).
- **A locally attractive behaviour that costs comfort:** in the last lecture hour (16:00–17:00, price ₹12, and nothing after it matters) the learned policy runs 3.28 kWh against 4.37 in the hour before, and accumulates 1.92 violation steps per day there, more than the thermostat (1.39). With these weights a peak-price High step costs about as much as a mild violation, so the agent accepts discomfort in the most expensive hour. This is the first sign that the comfort weight is the critical design parameter, which the attack below exploits.

Attack 1 – a reward that can be exploited (comfort weight too small)

Construction. Train with the comfort penalty reduced to $\lambda \in \{1, 3, 6\}$ ₹/°C ($\lambda_o = 0$), and compare with the design value ($\lambda = 15$, $\lambda_o = 5$), 8 seeds each. All controllers are then **judged with the designed metrics** on the same 200 test days. The loophole: cooling costs ₹9–18 per step, so if a violation is cheaper than that, "stay Off until the occupants have arrived and pay the penalty afterwards" has the higher return.

Prediction (written before running): $\lambda = 1$ → almost always Off, ≥ 5× the design violations; $\lambda = 3$ → Off before arrival and in the lunch gap, ≥ 2× the violations, lower cost.

Comfort weight λ used in training	Violation steps / day	Energy (kWh/day)	Cost (₹/day)	Reward under designed metric	Off share 06–09 h	Off share 13–14 h (lunch)	Violations 09–10 h	Violations 14–15 h
1	24.7	1.8	14	-2864	96%	99%	3.17	3.57
3	17.8	20.6	180	-1249	76%	57%	1.28	3.27
6	14.7	30.8	282	-627	75%	46%	1.46	2.72
15 (design)	4.1	36.5	352	-410	67%	42%	0.29	0.57

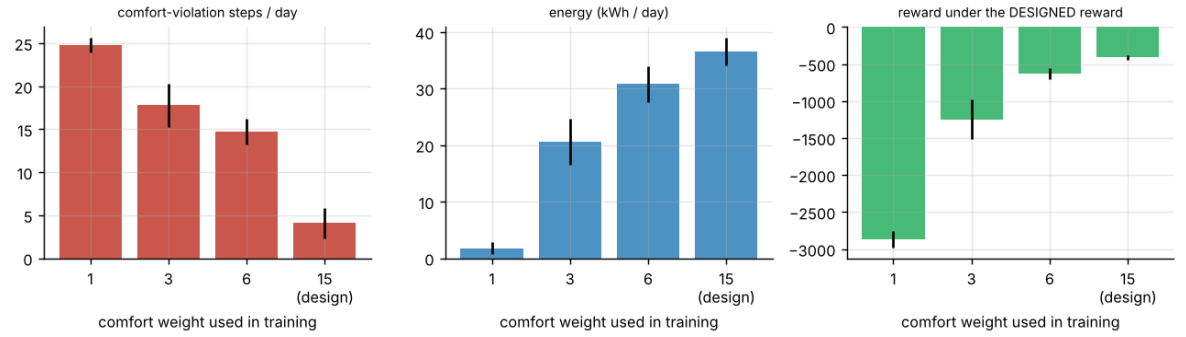


Figure 6.4 – Weak comfort weights buy lower energy with many more violations and a much worse designed reward.

What happened. Prediction confirmed in direction and size: $\lambda = 1$ → 24.7 violation steps/day (6.0× the design), energy 1.8 kWh – the controller is Off 96% of the time before lectures and 99% in the lunch gap, i.e. it has learned to do nothing; $\lambda = 3$ → 17.8 steps (4.3×), cost ₹180 vs ₹352. The *mechanism* is the one predicted – violations right after arrival (09–10 h: 0.29 → 3.17 steps; 14–15 h: 0.57 → 3.27) – but the "Off in the lunch gap" part is weaker than I predicted: at $\lambda = 3$ the Off share at lunch is 57%, only modestly above the design's 42%. **Which assumption failed:** the reward term for comfort. The agent maximised the return exactly as specified; with a small λ the specification itself rewards neglecting the occupants (reward hacking by under-specified penalty). The check that the weight is safe is simple: λ·(deviation of a typical short violation) must exceed the price of the cooling step that would avoid it (₹9–18 here); $\lambda = 1$–6 fails this, $\lambda = 15$ only just satisfies it – and the last-hour behaviour above shows even the design value leaves a loophole at peak price.

Attack 2 – unusual price pattern

Construction. Train on the normal tariff (policy frozen), then evaluate on days with a tariff spike of ₹30/kWh between 06:00 and 09:00 – exactly the pre-cooling window – and compare with a policy retrained on that tariff (8 seeds each, 200 test days). **Prediction (before running):** the frozen policy costs > 10 % more than the retrained policy, because its locally attractive "pre-cool right before the lecture" action was learned under a price that no longer holds.

Controller on spike-tariff days	Cost (₹/day)	Energy (kWh/day)	Violation steps/day	Reward	Energy 00–06 h (kWh)	Energy 06–09 h (kWh)
Frozen policy (trained on normal tariff)	440.2	35.9	3.4	-485	2.05	4.20
Policy retrained on spike tariff	366.9	37.1	3.7	-414	5.08	0.87
Scheduled thermostat	481.6	37.9	7.7	-540	–	–

What happened. Confirmed: the frozen policy costs **20.0 % more** than the retrained one (₹440 vs ₹367), because it still buys 4.2 kWh in the ₹30 window whereas the retrained policy buys 0.9 kWh there and moves

> its cooling to the night (5.1 kWh vs 2.1). To be fair, the frozen policy is not a disaster: it still undercuts the thermostat (₹440 vs ₹482), so the loss is regret, not collapse. **Which assumption failed:** a stationary environment – the state contains the *hour* but not the *price*, so the table cannot represent "the price in this hour has changed"; it only works because price was a fixed function of time during training. Repairs: add a price band to the state, or detect a tariff change and retrain/adapt online (a constant step size α helps adaptation).

Summary of the design defence

- **Does it work?** Yes, modestly: +8.6 % reward, −7 % energy, −12 % cost and fewer violation steps than the best fixed rule in 19 of 20 seeds, after about 3500 episodes.
- **When does it fail?** When the comfort weight is too small (Attack 1), when the tariff differs from training (Attack 2), and – at the design weights – in the last peak-price hour, where it trades comfort for money.
- **Why does it work?** Because the state exposes hour (price, timetable), temperature (thermal memory) and occupancy, TD learning propagates the cost of warm rooms at 09:00 back to cheap night-time cooling, which no fixed rule does.
- **What if the requirement changes?** A comfort *guarantee* instead of a trade-off needs a constraint (e.g. a large λ or a safety layer overriding Off when occupied and T > 26); a new tariff needs price in the state; a larger building needs function approximation instead of a table.

Overall summary, reflection and deliverables

Key results at a glance

Task	Main finding (numbers from the experiments above)
1	Training-set accuracy 1.000 vs held-out 0.825 for an unlimited tree on imbalanced data (optimism 17.5 points) while the majority baseline already scores 0.885.
2	Desk-check tie at every level (majority-of-three data); from-scratch tree agrees with scikit-learn on 97.7% of test predictions; reduced-error pruning cuts leaves 14 → 6 with no loss; modified row flips the root to Assignments; ID attribute gets maximal gain (test accuracy 0.481).
3	Missing normalisation of responsibilities: log-likelihood fell in 25/25 faulty runs, $\Sigma\pi \to \sim 10^{-35}$; repaired version never decreased it and matched scikit-learn; repair is neutral on inputs where the normaliser is 1.
4	Gradient check max relative error < 1e-8; perceptron ≤ 0.75 on XOR, MLP 20/20; 32-setting grid, best test accuracy 0.978; dying-ReLU failure (gradient norm exactly 0) and vanishing gradients (≈ 3000× across 7 layers) diagnosed.
5	Naive Bayes: smoothing matters when data are scarce ($\alpha = 0 \to$ 67/360 undefined rows); linear SVM at chance (0.506) vs RBF SVM 0.947; EBL rule is sound, covers 7/12 hazardous cases and is 2.5× cheaper on those, but adds no new entailed knowledge.
6	Q-learning beats the scheduled thermostat in 19/20 seeds (+8.6 % reward); weak comfort weights and a changed tariff both break it as predicted.

Predictions that were wrong or only partly right

- Task 4.6, P2: sigmoid at lr 0.01 did not under-fit for wide layers (0.78–0.82). P3: the ReLU lr = 1.0 collapse hit narrow/medium layers, not the widest. P4: the best learning rate was 1.0, not 0.1.
- Task 6: pre-cooling happened at night and at 08:00 rather than 06:00–08:00; “Off in the lunch gap” under a weak comfort weight was weaker than predicted.
- Task 2.7: I expected gain ratio to repair the identifier bias; it does so only for larger data, not for the 8-row table, and still builds 300 leaves deeper in the tree.

Limitations

- Data sets for Tasks 1 (set A), 5 and 6 are synthetic by necessity (no network access); conclusions about real students or real buildings would need real data.
- Several comparisons rest on one split with a few hundred test rows (Tasks 2.5, 2.6, 5.2); differences of one or two test cases are not significant and are flagged where they occur.
- The RL reward weights ($\lambda_o = 5$, $\lambda = 15$) were set from the cost scale of one peak-hour step and calibrated against the thermostat once, not tuned further; the attack shows the result depends on them.